

Evolução e desempenho do computador

2.1 Um breve histórico dos computadores

- A primeira geração: válvulas
- A segunda geração: transistores
- A terceira geração: circuitos integrados
- Gerações posteriores

2.2 Projetando visando desempenho

- Velocidade do microprocessador
- Balanço do desempenho
- Melhorias na organização e arquitetura do chip

2.3 Evolução da arquitetura Intel x86

2.4 Sistemas embarcados e o ARM

- Sistemas embarcados
- Evolução do ARM

2.5 Avaliação de desempenho

- Velocidade do clock e instruções por segundo
- Benchmarks
- Lei de Amdahl

2.6 Leitura recomendada e sites Web

PRINCIPAIS PONTOS

- A evolução dos computadores tem sido caracterizada pelo aumento na velocidade do processador, diminuição no tamanho do componente, aumento no tamanho da memória e aumento na capacidade e velocidade da E/S.
- Um fator responsável pelo grande aumento na velocidade do processador é o encolhimento no tamanho dos componentes do microprocessador; isso reduz a distância entre os componentes e, portanto, aumenta a velocidade. Contudo, os verdadeiros ganhos na velocidade nos anos recentes têm vindo da organização do processador, incluindo o uso intenso das técnicas de pipeline e execução paralela e do uso de técnicas de execução especulativas (tentativa de execução de instruções futuras que poderiam ser necessárias). Todas essas técnicas são projetadas para manter o processador ocupado pelo máximo de tempo possível.
- Uma questão crítica no projeto de sistema de computador é equilibrar o desempenho dos diversos elementos de modo que os ganhos no desempenho em uma área não sejam prejudicados por um atraso em outras áreas. Em particular, a velocidade do processador aumentou mais rapidamente do que o tempo de acesso da memória. Diversas técnicas são usadas para compensar essa divergência, incluindo caches, caminhos de dados mais largos da memória ao processador, e chips de memória mais inteligentes.

Vamos começar nosso estudo dos computadores com um breve histórico. O histórico por si só é interessante e também tem a finalidade de oferecer uma visão geral da estrutura e da função do computador. Em seguida, focalizamos a questão do desempenho. Uma consideração da necessidade pela utilização balanceada dos recursos do computador oferece um contexto que será útil por todo o livro. Finalmente, veremos rapidamente a evolução dos dois sistemas que servem como exemplo-chave no decorrer do livro: as famílias de processadores Intel x86 e ARM.



2.1 Um breve histórico dos computadores



A primeira geração: válvulas

ENIAC O ENIAC (*Electronic Numerical Integrator And Computer*), projetado e construído na Universidade da Pensilvânia, foi o primeiro computador digital eletrônico de uso geral do mundo. O projeto foi uma resposta às

necessidades dos EUA durante a Segunda Guerra Mundial. O *Ballistics Research Laboratory* (BRL) do Exército, uma agência responsável por desenvolver tabelas de faixa e trajetória para novas armas, estava tendo dificuldade para fornecer essas tabelas com precisão e dentro de um espaço de tempo razoável. Sem essas tabelas de disparo, as novas armas e artilharia eram inúteis aos artilheiros. O BRL empregou mais de 200 pessoas que, usando calculadoras de mesa, solucionavam as equações de balística necessárias. A preparação das tabelas para uma única arma exigiria muitas horas, até mesmo dias, de uma pessoa.

John Mauchly, professor de engenharia elétrica na Universidade da Pensilvânia, e John Eckert, um de seus alunos formados, propôs construir um computador de uso geral usando válvulas para a aplicação do BRL. Em 1943, o Exército aceitou essa proposta, e foi iniciado o trabalho no ENIAC. A máquina resultante era enorme, pesava 30 toneladas, ocupando 1 500 pés quadrados de superfície e contendo mais de 18 000 válvulas. Quando estava em operação, ela consumia 140 kilowatts de potência. Ela também era substancialmente mais rápida que qualquer computador eletromecânico, capaz de realizar 5 000 adições por segundo.

O ENIAC era uma máquina decimal, ao invés de binária. Ou seja, os números eram representados em formato decimal e a aritmética era realizada no sistema decimal. Sua memória consistia em 20 “acumuladores”, cada um capaz de manter um número decimal de 10 dígitos. Um anel de 10 válvulas representava cada dígito. A qualquer momento, somente uma válvula estava no estado LIGADO, representando um dos 10 dígitos. A principal desvantagem do ENIAC foi que ele tinha que ser programado manualmente, por meio da ligação de chaves e conexão e desconexão de cabos.

O ENIAC foi concluído em 1946, muito tarde para ser usado no esforço da guerra. Em vez disso, sua primeira tarefa foi realizar uma série de cálculos complexos que foram usados para ajudar a determinar a viabilidade da bomba de hidrogênio. O uso do ENIAC para um propósito diferente daquele para o qual foi construído demonstrou sua natureza de uso geral. O ENIAC continuou a operar sob a gerência do BRL até 1955, quando foi desmontado.

A MÁQUINA DE VON NEUMANN A tarefa de entrar e alterar programas para o ENIAC era extremamente enfadonha. O processo de programação poderia ser facilitado se o programa pudesse ser representado em uma forma adequada para armazenamento na memória junto com os dados. Então, um computador poderia obter suas instruções lendo-as da memória, e um programa poderia ser criado ou alterado definindo-se os valores de uma parte da memória.

Essa ideia, conhecida como *conceito de programa armazenado*, normalmente é atribuída aos projetistas do ENIAC, principalmente o matemático John von Neumann, que foi consultor no projeto ENIAC. Alan Turing desenvolveu a ideia praticamente ao mesmo tempo. A primeira publicação da ideia foi em uma proposta de 1945 de von Neumann para um novo computador, o EDVAC (*Electronic Discrete Variable Computer*).

Em 1946, von Neumann e seus colegas começaram o projeto de um novo computador de programa armazenado, conhecido como computador IAS, no *Princeton Institute for Advanced Studies*. O computador IAS, embora não concluído antes de 1952, é o protótipo de todos os computadores de uso geral.

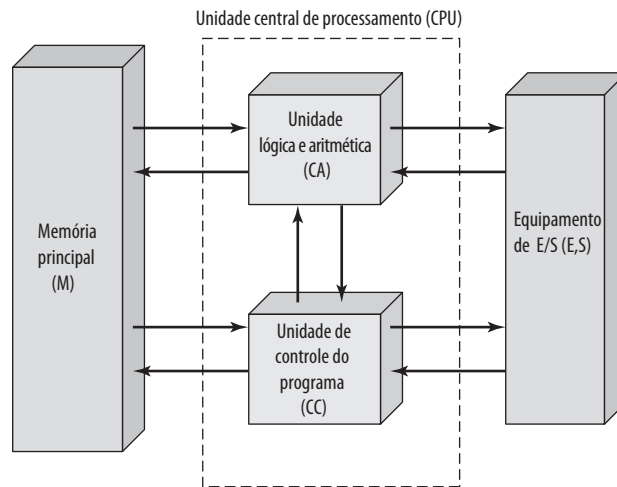
A Figura 2.1 mostra a estrutura geral de um computador IAS (compare com a parte do meio da Figura 1.4). Ela consiste em:

- Uma memória principal, que armazena dados e instruções.¹
- Uma unidade lógica e aritmética (ALU) capaz de operar sobre dados binários.
- Uma unidade de controle, que interpreta as instruções na memória e faz com que sejam executadas.
- Equipamento de entrada e saída (E/S) operado pela unidade de controle.

Essa estrutura foi esboçada na proposta inicial de von Neumann, que vale a pena citar neste ponto (VON Neumann, 1993^a):

2.2 Primeiro: como o dispositivo é principalmente um computador, ele terá que realizar as operações elementares da aritmética mais frequentemente. São elas adição, subtração, multiplicação e divisão. Portanto, é razoável que ele contenha unidades especializadas apenas para essas operações.

¹ Neste livro, a menos que observado de outra forma, o termo *instrução* refere-se a uma instrução de máquina que é interpretada e executada diretamente pelo processador, ao contrário de uma instrução em uma linguagem de alto nível, como Ada ou C++, que primeiro precisa ser compilada para uma série de instruções de máquina antes que seja executada.

Figura 2.1 Estrutura de um computador IAS

Deve-se observar, porém, que embora esse princípio como tal provavelmente seja seguro, o modo específico em que é realizado exige uma análise mais criteriosa. Em qualquer velocidade, uma parte *aritmética central* do dispositivo provavelmente terá que existir, e isso constitui a *primeira parte específica*: CA (do inglês **Central Arithmetic**).

2.3 **Segundo:** o controle lógico do dispositivo, ou seja, a sequenciação apropriada de suas operações, pode ser executado de forma mais eficiente por um órgão de controle central. Se o dispositivo tiver que ser *flexível*, ou seja, de *propósito geral*, então deve-se distinguir entre as instâncias específicas dadas para isso, definindo um problema em particular e as unidades gerais de controle que fazem essas instruções — não importa quais sejam — serem executadas. O primeiro deverá ser armazenado de alguma maneira; o segundo é representado por partes operacionais definidas do dispositivo. Por *controle central*, queremos dizer apenas essa última função, e as unidades que o realizam formam a *segunda parte específica*: CC.

2.4 **Terceiro:** qualquer dispositivo que tiver que executar sequências de operações longas e complicadas (especificamente de cálculos) precisa ter uma memória considerável [...]

(b) As instruções que controlam um problema complicado podem constituir um material considerável, principalmente se o código for circunstancial (que é na maioria dos arranjos). Esse material precisa ser lembrado.

Em qualquer velocidade, a *memória total* constitui a *terceira parte específica do dispositivo*: M.

2.6 As três partes específicas, CA, CC e M, correspondem aos neurônios *associativos* no sistema nervoso humano. Ainda falta discutir os equivalentes dos neurônios *sensoriais* ou *aferentes* e os neurônios *motores* ou *eferentes*. Estas são as unidades de *entrada* e *saída* do dispositivo.

O dispositivo precisa ser capaz de manter contato de entrada e saída (sensorial e motor) com algum meio específico desse tipo. O meio será chamado de *meio de gravação exterior do dispositivo*: R (do inglês **recording**).

2.7 **Quarto:** o dispositivo precisa ter unidades para transferir [...] informações de R para suas partes específicas C e M. Essas unidades formam sua *entrada*, a *quarta parte específica*: I (do inglês *input*). Veremos que é melhor fazer todas as transferências de R (por I) para M, e nunca diretamente de C.

2.8 **Quinto:** o dispositivo precisa ter unidades para transferir [...] de suas partes específicas C e M para R. Essas unidades formam sua *saída*, a *quinta parte específica*: O (do inglês *output*). Veremos novamente que é melhor fazer todas as transferências de M (por O) para R, e nunca diretamente de C.

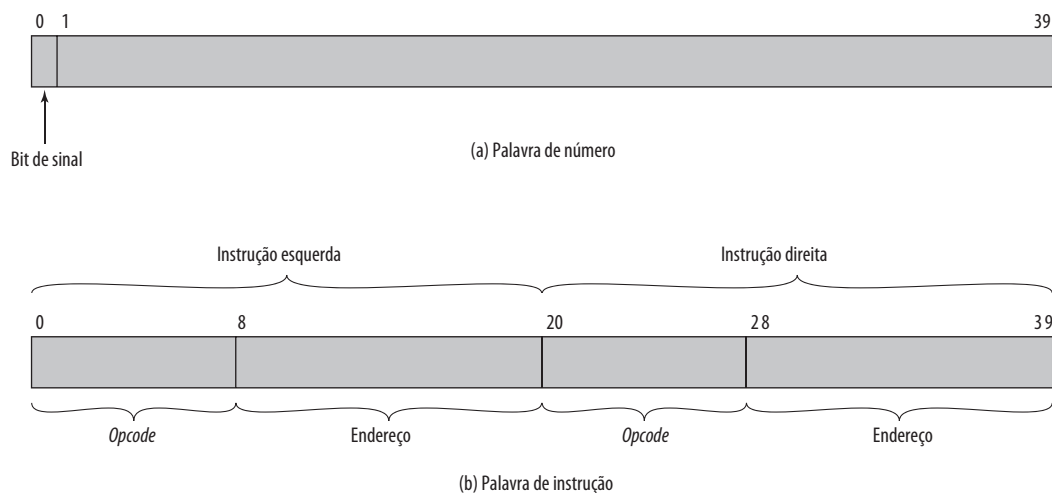
Com raras exceções, todos os computadores de hoje têm essa mesma estrutura e função geral, e são conhecidos como máquinas de von Neumann. Assim, vale a pena, neste ponto, descrever rapidamente a operação do computador IAS (Burks, 1946^b). Após Hayes (1998^c), a terminologia e a notação de Von Neumann são alteradas como a seguir para conformar mais de perto com o uso moderno; os exemplos e as ilustrações que acompanham esta discussão são baseados nesse último texto.

A memória do IAS consiste em 1 000 locais de armazenamento, chamados *palavras (words)*, de 40 dígitos binários (bits) cada.² Tanto dados quanto instruções são armazenados lá. Números são representados em formato binário e cada instrução é um código binário. A Figura 2.2 ilustra esses formatos. Cada número é representado por um bit de sinal e um valor de 39 bits. Uma palavra também pode conter duas instruções de 20 bits, com cada instrução consistindo em um código de operação de 8 bits (*opcode*), especificando a operação a ser realizada, e um endereço de 12 bits, designando uma das palavras na memória (numeradas de 0 a 999).

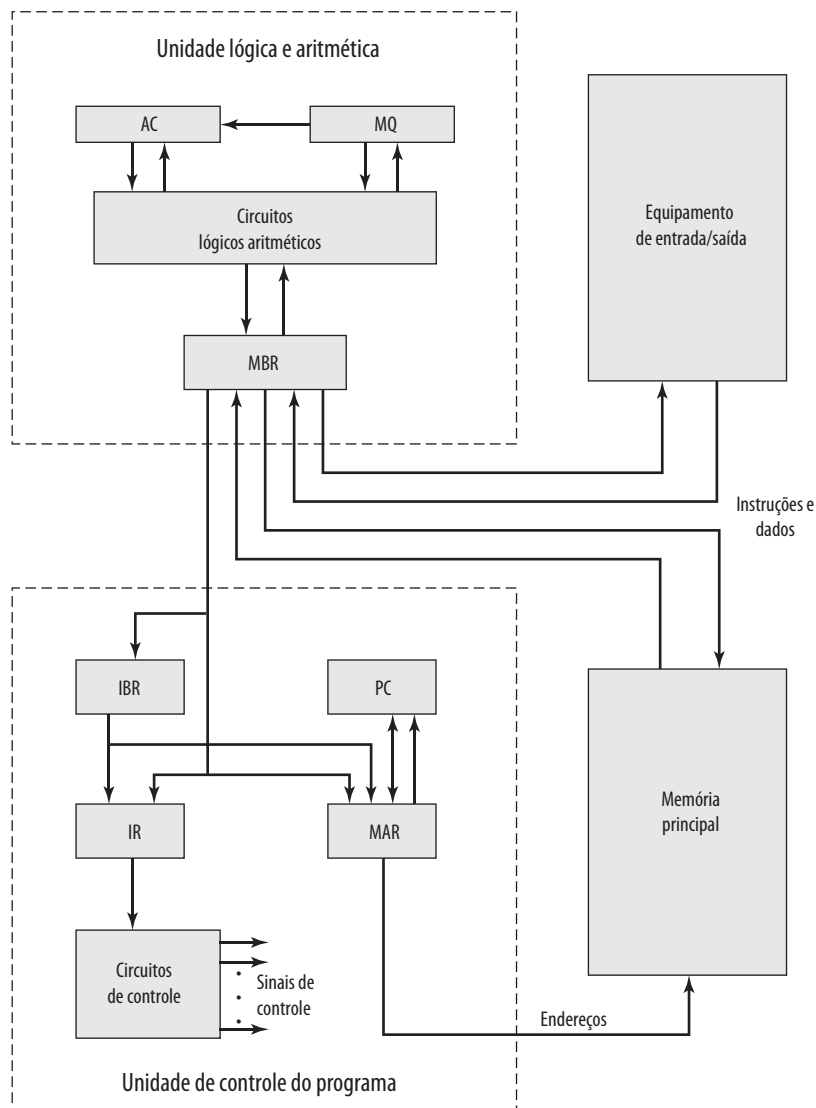
A unidade de controle opera o IAS buscando instruções da memória e executando-as uma de cada vez. Para explicar isso, um diagrama de estrutura mais detalhado é necessário, conforme indicado na Figura 2.3. Essa figura revela que a unidade de controle e a ALU contêm locais de armazenamento, chamados *registradores*, definidos da seguinte forma:

- **Registrador de buffer de memória (MBR, do inglês *memory buffer register*)**: contém uma palavra a ser armazenada na memória ou enviada à unidade de E/S, ou é usada para receber uma palavra da memória ou de uma unidade de E/S.
- **Registrador de endereço de memória (MAR, do inglês *memory address register*)**: especifica o endereço na memória da palavra a ser escrita ou lida no MBR.
- **Registrador de instrução (IR, do inglês *instruction register*)**: contém o *opcode* de 8 bits da instrução que está sendo executada.
- **Registrador de buffer de instrução (IBR, do inglês *instruction buffer register*)**: empregado para manter temporariamente a próxima instrução a ser executada.
- **Contador de programa (PC, do inglês *program counter*)**: contém o endereço do próximo par de instruções a ser apanhado da memória.

Figura 2.2 Formatos de memória do IAS

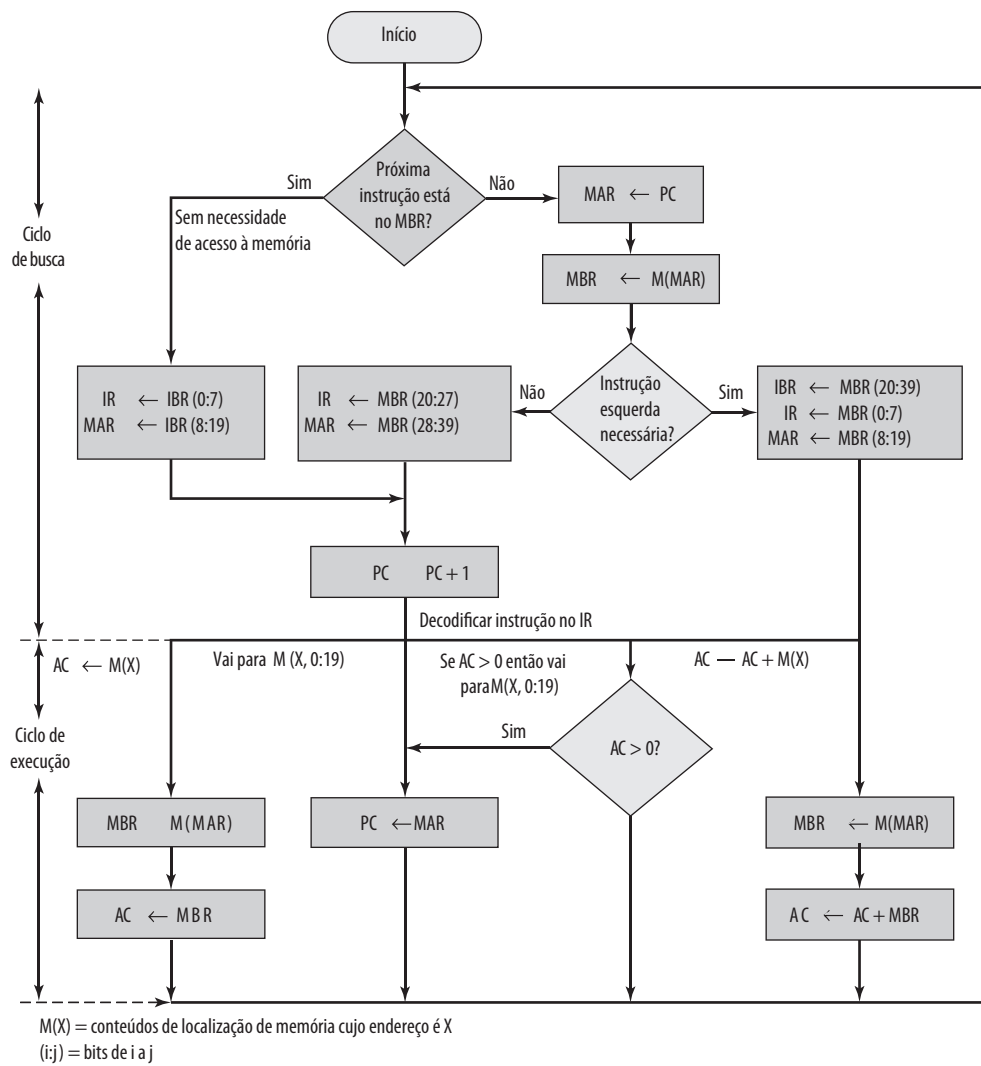


² Não existe uma definição universal para o termo palavra. Em geral, uma palavra é um conjunto ordenado de bytes ou bits, que é a unidade normal na qual a informação pode ser armazenada, transmitida ou operada dentro de determinado computador. Normalmente, se um processador tem um conjunto de instruções de tamanho fixo, então o tamanho da instrução é igual ao tamanho da palavra.

Figura 2.3 Estrutura expandida do computador IAS

- **Acumulador (AC) e quociente multiplicador (MQ, do inglês *multiplier quotient*):** empregado para manter temporariamente operandos e resultados de operações da ALU. Por exemplo, o resultado de multiplicar dois números de 40 bits é um número de 80 bits; os 40 bits mais significativos são armazenados no AC e o menos significativos no MQ.

O IAS opera realizando repetidamente um *ciclo de instrução*, como mostra a Figura 2.4. Cada ciclo de instrução consiste em dois subciclos. Durante o *ciclo de busca (fetch cycle)*, o *opcode* da próxima instrução é carregado no IR e a parte de endereço é carregada no MAR. Essa instrução pode ser retirada do IBR ou pode ser obtida da memória carregando-se uma palavra no MBR, e depois para o IBR, IR e MAR.

Figura 2.4 Fluxograma parcial da operação do IAS

Por que a indireção? Essas operações são controladas por circuitos eletrônicos e resultam no uso de caminhos de dados. Para simplificar a eletrônica, somente um registrador é usado para especificar o endereço na memória para uma leitura ou escrita e somente um registrador é usado para a origem ou o destino.

Quando o *opcode* está no IR, o *ciclo de execução* é realizado. O circuito de controle interpreta o *opcode* e executa a instrução enviando os sinais de controle apropriados para que os dados sejam movidos ou uma operação seja realizada pela ALU.

O computador IAS tinha um total de 21 instruções, listadas na Tabela 2.1. Estas podem ser agrupadas da seguinte forma:

- **Transferência de dados:** movem dados entre memória e registradores da ALU ou entre dois registradores da ALU.
- **Desvio incondicional:** normalmente, a unidade de controle executa instruções em sequência a partir da memória. Essa sequência pode ser alterada por uma instrução de desvio, que facilita operações repetitivas.
- **Desvio condicional:** o desvio pode se tornar dependente de uma condição, permitindo assim pontos de decisão.
- **Aritméticas:** operações realizadas pela ALU.

- **Modificação de endereço:** permite que os endereços sejam calculados na ALU e depois inseridos em instruções armazenadas na memória. Isso permite a um programa uma flexibilidade de endereçamento considerável.

A Tabela 2.1 apresenta as instruções em um formato simbólico, de fácil leitura. Na realidade, cada instrução precisa estar de acordo com o formato da Figura 2.2b. A parte do *opcode* (primeiros 8 bits) especifica qual das 21 instruções deve ser executada. A parte de endereço (12 bits restantes) especifica qual dos 1.000 locais de memória deverá estar envolvido na execução da instrução.

A Figura 2.4 mostra vários exemplos de execução de instrução pela unidade de controle. Observe que cada operação requer várias etapas, algumas muito complicadas. A operação de multiplicação requer 39 suboperações, uma para cada posição de bit, exceto o bit de sinal.

COMPUTADORES COMERCIAIS A década de 1950 viu o nascimento da indústria do computador com duas empresas, Sperry e IBM, dominando o mercado.

Tabela 2.1 O conjunto de instruções do IAS

Tipo de instrução	Opcode	Representação simbólica	Descrição
Transferência de dados	00001010	LOAD MQ	Transfere o conteúdo de MQ para AC
	00001001	LOAD MQ,M(X)	Transfere o conteúdo do local de memória X para MQ
	00100001	STOR M(X)	Transfere o conteúdo de AC para o local de memória X
	00000001	LOAD M(X)	Transfere M(X) para o AC
	00000010	LOAD – M(X)	Transfere – M(X) para o AC
	00000011	LOAD M(X)	Transfere o valor absoluto de M(X) para o AC
	00000100	LOAD – M(X)	Transfere – M(X) para o acumulador
Desvio incondicional	00001101	JUMP M(X,0:19)	Apanha a próxima instrução da metade esquerda de M(X)
	00001110	JUMP M(X,20:39)	Apanha a próxima instrução da metade direita de M(X)
Desvio condicional	00001111	JUMP+ M(X,0:19)	Se o número no AC for não negativo, apanha a próxima instrução da metade esquerda de M(X)
	00010000	JUMP+ M(X,20:39)	Se o número no AC for não negativo, apanha a próxima instrução da metade direita de M(X)
Aritmética	00000101	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000111	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000110	SUB M(X)	Subtrai M(X) de AC; coloca o resultado em AC
	00001000	SUB M(X)	Subtrai M(X) de AC; coloca o resto em AC
	00001011	MUL M(X)	Multiplica M(X) por MQ; coloca os bits mais significativos do resultado em AC; coloca bits menos significativos em MQ
	00001100	DIV M(X)	Divide AC por M(X); coloca o quociente em MQ e o resto em AC
	00010100	LSH	Multiplica o AC por 2; ou seja, desloca à esquerda uma posição de bit
	00010101	RSH	Divide o AC por 2; ou seja, desloca uma posição à direita
Modificação de endereço	00010010	STOR M(X,8:19)	Substitui campo de endereço da esquerda em M(X) por 12 bits mais à direita de AC
	00010011	STOR M(X,28:39)	Substitui campo de endereço da direita em M(X) por 12 bits mais à direita de AC

Em 1947, Eckert e Mauchly formaram a Eckert-Mauchly Computer Corporation para manufaturar computadores comercialmente. Sua primeira máquina de sucesso foi o UNIVAC I (*Universal Automatic Computer*), que foi comissionado pelo Birô do Censo dos Estados Unidos da América para os cálculos de 1950. A Eckert-Mauchly Computer Corporation tornou-se parte da divisão UNIVAC da Sperry-Rand Corporation, que construiu uma série de máquinas sucessoras.

O UNIVAC I foi o primeiro computador comercial de sucesso. Ele tinha como finalidade aplicações científicas e comerciais. O primeiro artigo descrevendo o sistema listava cálculos algébricos de matriz, problemas estatísticos, cobranças de prêmio para uma companhia de seguros e problemas de logística como exemplos das tarefas que ele poderia realizar.

O UNIVAC II, que tinha maior capacidade de memória e maior desempenho que o UNIVAC I, foi entregue no final da década de 1950 e ilustrava várias tendências que permaneceram características da indústria de computação. Primeiro, os avanços na tecnologia permitiram que as empresas continuassem a construir computadores maiores e mais poderosos. Segundo, cada empresa tentou tornar suas novas máquinas *compatíveis*³ com as máquinas mais antigas, ou os programas escritos para as máquinas mais antigas poderiam ser executados nas novas máquinas. Essa estratégia foi adotada na esperança de reter a base de clientes; ou seja, quando um cliente decide comprar uma nova máquina, ele provavelmente a recebe da mesma empresa, para evitar perder o investimento nos programas.

A divisão UNIVAC também iniciou o desenvolvimento de computadores da série 1100, que deveria ser sua principal fonte de receita. Essa série ilustra uma distinção que existia na época. O primeiro modelo, o UNIVAC 1103 e seus sucessores, por muitos anos, foram voltados principalmente para aplicações científicas, envolvendo cálculos grandes e complexos. Outras empresas se concentraram em aplicações comerciais, que envolviam grandes quantidades de dados de texto. Essa separação, em grande parte, desapareceu, mas ficou evidente por muitos anos.

A IBM, então o principal fabricante de equipamento de processamento de cartão perfurado, entregou seu primeiro computador de programa armazenado, o 701, em 1953. O 701 era voltado principalmente para aplicações científicas (Bashe et al., 1981^o). Em 1955, a IBM introduziu o produto 702, que tinha uma série de recursos de hardware que o capacitavam para aplicações comerciais. Estes foram os primeiros de uma longa série de computadores 700/7000, que estabeleceram a IBM como o fabricante de computadores esmagadoramente dominante.



A segunda geração: transistores

A primeira mudança importante no computador eletrônico veio com a substituição da válvula pelo transistor. O transistor é menor, mais barato e dissipa menos calor que uma válvula, mas pode ser usado da mesma forma que uma válvula para construir computadores. Diferente da válvula, que exige fios, placas de metal, uma cápsula de vidro e um vácuo, o transistor é um *dispositivo de estado sólido*, feito de silício.

O transistor foi inventado na Bell Laboratórios em 1947 e, por volta da década de 1950, deu início a uma revolução eletrônica. Porém, não foi antes da década de 1950 que os computadores transistorizados foram disponibilizados comercialmente. A IBM novamente foi a primeira empresa a oferecer a nova tecnologia. A NCR e, com mais sucesso, a RCA foram as pioneiras com algumas máquinas pequenas a transistor. A IBM veio pouco depois com a série 7000.

O uso do transistor define a *segunda geração* de computadores. Foi bastante aceito classificar os computadores em gerações com base na tecnologia de hardware fundamental empregada (Tabela 2.2). Cada nova geração é caracterizada por maior desempenho de processamento, maior capacidade de memória e tamanho menor que a anterior.

Mas também existem outras mudanças. A segunda geração viu a introdução de unidades lógicas e aritméticas e unidades de controle mais complexas, o uso de linguagens de programação de alto nível e a disponibilidade do *software de sistema* com o computador.

3 Também chamadas *compatíveis para trás* (*downward compatible*). O mesmo conceito, do ponto de vista do sistema mais antigo, é conhecido como *compatível para frente* (*forward compatible*).

Tabela 2.2 Gerações de computador

Geração	Datas aproximadas	Tecnologia	Velocidade típica (operações por segundo)
1	1946 – 1957	Válvula	40.000
2	1958 – 1964	Transistor	200.000
3	1965 – 1971	Integração em escala pequena e média	1.000.000
4	1972 – 1977	Integração em escala grande	10.000.000
5	1978 – 1991	Integração em escala muito grande	100.000.000
6	1991-	Integração em escala ultragrande	1.000.000.000

A segunda geração é digna de nota também pelo surgimento da *Digital Equipment Corporation* (DEC). A DEC foi fundada em 1957 e, nesse ano, entregou seu primeiro computador, o PDP-1. Esse computador e essa empresa iniciaram o fenômeno do minicomputador, que se tornaria tão proeminente na terceira geração.

O IBM 7094 Desde a introdução da série 700 em 1952 até a introdução do último membro da série 7000 em 1964, essa linha de produtos IBM passou por uma evolução que é típica dos produtos de computação. Os membros sucessivos da linha de produtos mostram desempenho maior, capacidade maior e/ou menor custo.

A Tabela 2.3 ilustra essa tendência. O tamanho da memória principal, em múltiplos de 2^{10} palavras de 36 bits, cresceu de 2K ($1K = 2^{10}$) para 32K palavras,⁴ enquanto o tempo para acessar uma palavra de memória, *tempo de ciclo de memória*, caiu de 30 μs para 1,4 μs . O número de *opcodes* cresceu de modestos 24 para 185.

A última coluna indica a velocidade de execução relativa da CPU. As melhorias de velocidade são alcançadas pela eletrônica melhorada (por exemplo, uma implementação de transistor é mais rápida que uma implementação de válvula) e circuitos mais complexos. Por exemplo, o IBM 7094 inclui um *Instruction Backup Register*, usado para armazenar a próxima instrução. A unidade de controle apanha duas palavras adjacentes da memória para uma busca de instrução. Exceto pela ocorrência de uma instrução de desvio, que normalmente é pouco frequente, isso significa que a unidade de controle precisa acessar a memória para uma instrução em apenas metade dos ciclos de instrução. Essa pré-busca reduz significativamente o tempo médio do ciclo de instrução.

Tabela 2.3 Exemplo de membros da série IBM 700/7000

Número do modelo	Primeira entrega	Tecnologia de CPU	Tecnologia de memória	Tempo de ciclo (μs)	Tamanho de memória (K)	Número de <i>opcodes</i>	Número de registradores de índice	Ponto flutuante hardwired	E/S sobreposta (canais)	Sobreposição de busca de instrução	Velocidade (relativa ao 701)
701	1952	Válvulas	Tubos eletrostáticos	30	2–4	24	0	não	não	não	1
704	1955	Válvulas	Core	12	4–32	80	3	sim	não	não	2,5
709	1958	Válvulas	Core	12	32	140	3	sim	sim	não	4
7090	1960	Transistor	Core	2,18	32	169	3	sim	sim	não	25
7094 I	1962	Transistor	Core	2	32	185	7	sim (precisão dupla)	sim	sim	30
7094 II	1964	Transistor	Core	1,4	32	185	7	sim (precisão dupla)	sim	sim	50

4 Uma discussão dos usos dos prefixos numéricos, como kilo e giga, pode ser encontrada em um documento de suporte no *Computer Science Student Resource Site*, disponível em <WilliamStallings.com/StudentSupport.html>.

O restante das colunas da Tabela 2.3 se tornará claro com a continuação do texto.

A Figura 2.5 mostra uma configuração grande (muitos periféricos) de um IBM 7094, que representa os computadores de segunda geração (Bell, 1971^e). Várias diferenças do computador IAS merecem ser mencionadas. A mais importante delas é o uso de *canais de dados*. Um canal de dados é um módulo de E/S independente, com seu próprio processador e seu próprio conjunto de instruções. Em um sistema de computador com esses dispositivos, a CPU não executa completamente instruções de E/S. Essas instruções são armazenadas em uma memória principal para serem executadas por um processador de uso específico no próprio canal de dados. A CPU inicia uma transferência de E/S enviando um sinal de controle ao canal de dados, instruindo-o a executar uma sequência de instruções na memória. O canal de dados realiza sua tarefa independentemente da CPU e sinaliza à CPU quando a operação termina. Esse arranjo libera a CPU de um peso considerável de processamento.

Outro recurso novo é o *multiplexador*, que é o ponto de término central para os canais de dados, a CPU e a memória. O multiplexador escalona o acesso à memória da CPU e dos canais de dados, permitindo que esses dispositivos atuem independentemente.

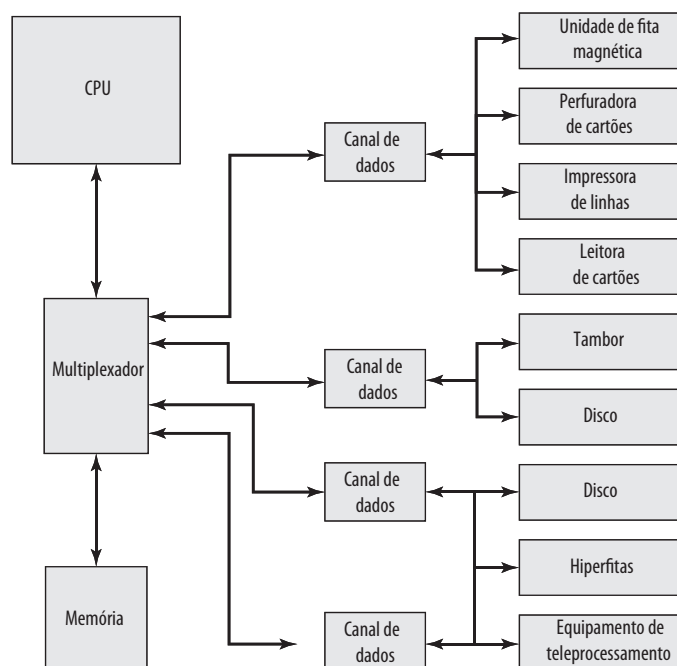


A terceira geração: circuitos integrados

Um transistor isolado, autocontido, é chamado de *componente discreto*. Pelos anos 1950 e início dos anos 1960, o equipamento eletrônico era composto principalmente de componentes discretos — transistores, resistores, capacitores e assim por diante. Os componentes discretos eram fabricados separadamente, empacotados em seus próprios invólucros e soldados ou ligados em placas de circuito tipo masonite, que eram então instaladas nos computadores, osciloscópios e outros equipamentos eletrônicos. Sempre que um equipamento eletrônico exigia um transistor, um pequeno tubo de metal, contendo uma peça de silício do tamanho de uma cabeça de alfinete, tinha que ser soldado a uma placa de circuito. O processo de manufatura inteiro, do transistor à placa de circuito, era dispendioso e complicado.

Esses fatos da vida estavam começando a criar problemas na indústria do computador. Os primeiros computadores de segunda geração continham cerca de 10 000 transistores. Esse número cresceu para centenas de milhares, tornando a manufatura de máquinas mais novas e mais poderosas cada vez mais difícil.

Figura 2.5 Configuração de um IBM 7094



Em 1958, chegou a realização que revolucionou a eletrônica e iniciou a era da microeletrônica: a invenção do circuito integrado, que define a terceira geração de computadores. Nesta seção, oferecemos uma breve introdução à tecnologia dos circuitos integrados. Depois, examinamos talvez os dois membros mais importantes da terceira geração, ambos introduzidos no início dessa era: o IBM System/360 e o DEC PDP-8.

MICROELETRÔNICA Microeletrônica significa literalmente “pequena eletrônica”. Desde os primórdios da eletrônica digital e da indústria da computação, tem havido uma tendência persistente e consistente em direção à redução no tamanho dos circuitos eletrônicos digitais. Antes de examinarmos as implicações e os benefícios dessa tendência, precisamos dizer algo sobre a natureza da eletrônica digital. Uma discussão mais detalhada pode ser encontrada no Capítulo 20.

Os elementos básicos de um computador digital, como sabemos, precisam realizar funções de armazenamento, movimentação, processamento e controle. Somente dois tipos fundamentais de componentes são necessários (Figura 2.6): portas e células de memória. Uma porta é um dispositivo que implementa uma função booleana ou lógica simples, como IF $A \text{ AND } B \text{ ARE TRUE THEN } C \text{ IS TRUE}$ (porta AND). Esses dispositivos são chamados de portas porque controlam o fluxo de dados de modo semelhante às portas de canal. A célula de memória é um dispositivo que pode armazenar um bit de dados; ou seja, o dispositivo pode estar em um de dois estados estáveis de cada vez. Interconectando grandes quantidades desses dispositivos fundamentais, podemos construir um computador. Podemos relacionar isso com nossas quatro funções básicas da seguinte forma:

- **Armazenamento de dados:** fornecido por células de memória.
- **Processamento de dados:** fornecido por portas.
- **Movimentação de dados:** os caminhos entre os componentes são usados para movimentar dados da memória para a memória e da memória pelas portas até a memória.
- **Controle:** os caminhos entre os componentes podem transportar sinais de controle. Por exemplo, uma porta terá uma ou duas entradas de dados mais uma entrada de sinal de controle que ativa a porta. Quando o sinal de controle é ON, a porta realiza sua função sobre as entradas de dados e produz uma saída de dados. De modo semelhante, a célula de memória armazenará o bit que está em seu fio de entrada quando o sinal de controle ESCRITA for ON, e colocará o bit que está na célula em seu fio de saída quando o sinal de controle LEITURA for ON.

Assim, um computador consiste em portas, células de memória e interconexões entre esses elementos. As portas e células de memória são, por sua vez, construídas de componentes eletrônicos digitais simples.

O circuito integrado explora o fato de que componentes como transistores, resistores e condutores podem ser fabricados a partir de um semicondutor como o silício. Essa é uma extensão da arte do estado sólido para fabricar um circuito inteiro em um pequeno pedaço de silício, ao invés de montar componentes discretos, feitos de partes separadas de silício no mesmo circuito. Muitos transistores podem ser produzidos ao mesmo tempo em um único wafer de silício. Igualmente importantes, esses transistores podem ser conectados com um processo de metalização para formar circuitos.

A Figura 2.7 representa os principais conceitos de um circuito integrado. Um *wafer* fino de silício é dividido em uma matriz de pequenas áreas, cada uma com poucos milímetros quadrados. O padrão de circuito idêntico é fabricado em cada área, e o wafer é dividido em *chips*. Cada chip consiste em muitas portas e/ou células de memória mais uma série de pontos de conexão de entrada e saída. Esse chip é então empacotado em um invólucro

Figura 2.6 Elementos fundamentais do computador

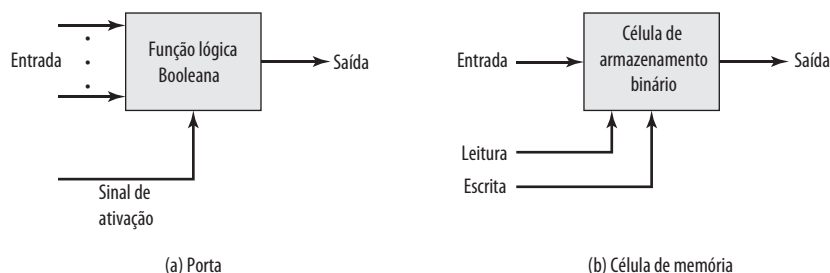
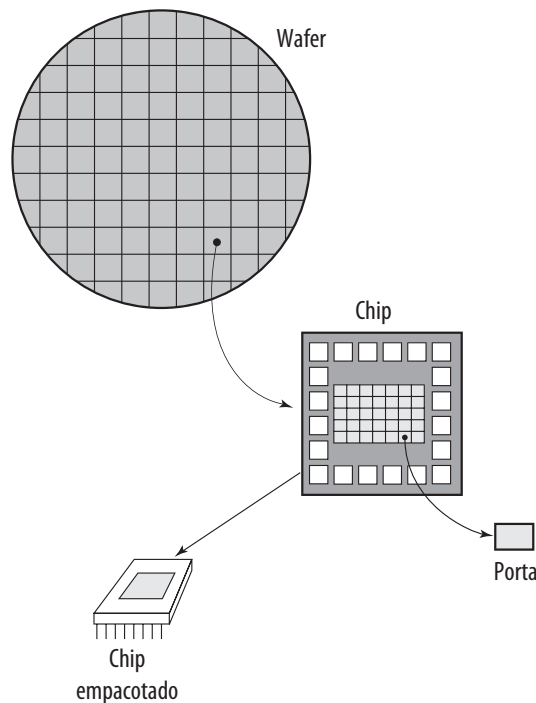


Figura 2.7 Relacionamento entre wafer, chip e porta

que o protege e oferece pinos para conexão com dispositivos além do chip. Diversos desses pacotes podem então ser interconectados em uma placa de circuito impresso para produzir circuitos maiores e mais complexos.

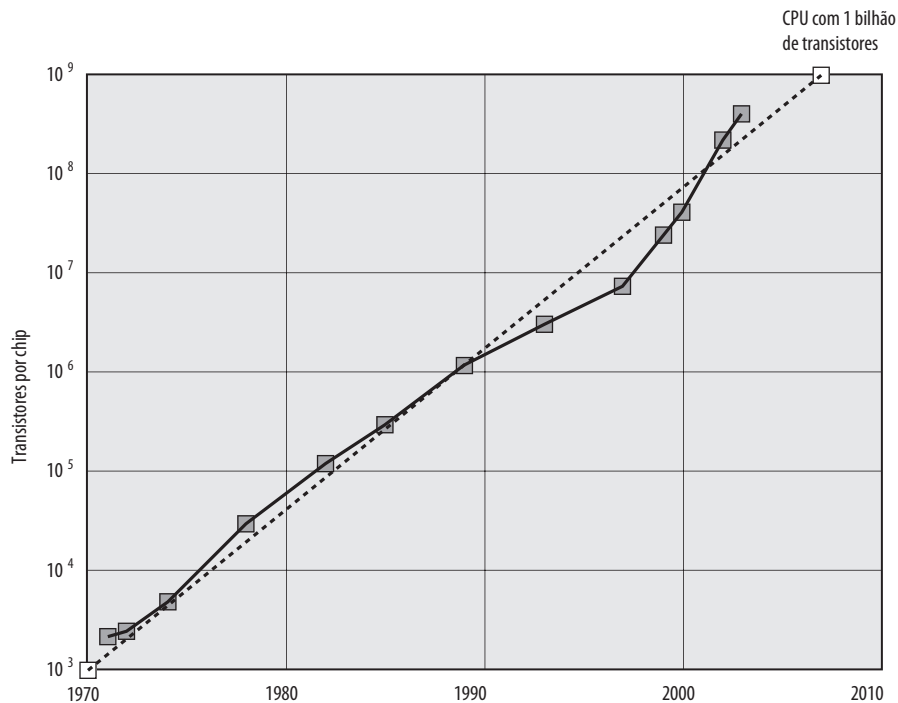
Inicialmente, somente algumas portas ou células de memória poderiam ser confiavelmente manufaturadas e empacotadas. Esses primeiros circuitos integrados são conhecidos como *integração em pequena escala* (SSI, do inglês *small-scale integration*). Com o passar do tempo, foi possível empacotar mais e mais componentes no mesmo chip. Esse crescimento na densidade é ilustrado na Figura 2.8; essa é uma das tendências tecnológicas mais marcantes já registradas.⁵ Essa figura reflete a famosa lei de Moore, que foi proposta por Gordon Moore, cofundador da Intel, em 1965 (Moore, 1965^f). Moore observou que o número de transistores que poderia ser colocado em um único chip estava dobrando a cada ano e previu corretamente que esse ritmo continuaria no futuro próximo. Para a surpresa de muitos, incluindo Moore, o ritmo continuou ano após ano e década após década. O ritmo diminuiu para dobrar a cada 18 meses na década de 1970, mas sustentou essa taxa desde então.

As consequências da lei de Moore são profundas:

1. O custo de um chip permaneceu praticamente inalterado durante esse período de rápido crescimento em densidade. Isso significa que o custo da lógica do computador e do circuito de memória caiu em uma taxa considerável.
2. Como os elementos lógicos e da memória são colocados muito próximos em chips mais densamente empacotados, a extensão do caminho elétrico é encurtada, aumentando a velocidade de operação.
3. O computador torna-se menor, fazendo com que seja mais conveniente colocá-lo em diversos ambientes.
4. Há uma redução nos requisitos de potência e resfriamento.
5. As interconexões no circuito integrado são muito mais confiáveis do que as conexões de solda. Com mais circuitos em cada chip, existem menos conexões entre chips.

IBM SYSTEM/360 Por volta de 1964, a IBM tinha uma firme compreensão do mercado de computador, com sua série de máquinas 7000. Nesse ano, ela anunciou o System/360, uma nova família de produtos de computador. Embora

⁵ Observe que o eixo vertical usa uma escala logarítmica. Uma revisão básica das escalas logarítmicas está no documento de revisão de matemática no *Computer Science Student Support Site*, disponível em <WilliamStallings.com/StudentSupport.html>.

Figura 2.8 Crescimento na contagem de transistores da CPU (BOHR, 2003⁹)

o anúncio em si não tenha sido surpresa, ele continha algumas notícias desagradáveis para os atuais clientes da IBM: a linha de produtos 360 era incompatível com as máquinas IBM mais antigas. Assim, a transição para o 360 seria difícil para a base de clientes atual. Esse foi um passo corajoso para a IBM, mas a empresa o achou necessário para romper algumas das restrições da arquitetura 7000 e produzir um sistema capaz de evoluir com a nova tecnologia de circuito integrado (Padegs, 1981^h; Gifford, 1987ⁱ). A estratégia compensou financeira e tecnicamente. O 360 foi o sucesso da década e concretizou a IBM como o fornecedor de computadores dominante, com uma fatia de mercado de mais de 70%. E, com algumas modificações e extensões, a arquitetura do 360 permanece até hoje na arquitetura dos computadores mainframe⁶ da IBM. Alguns exemplos usando essa arquitetura podem ser encontrados no decorrer deste texto.

O System/360 foi a primeira família de computadores planejada do setor. A família cobria uma grande faixa de desempenho e custo. A Tabela 2.4 indica algumas das principais características dos diversos modelos em 1965 (cada membro da família é distinguido por um número de modelo). Os modelos eram compatíveis no sentido de que um programa escrito para um modelo deveria ser capaz de ser executado por outro modelo da série, com diferença apenas no tempo gasto para executar.

O conceito de uma família de computadores compatíveis foi moderno e extremamente bem-sucedido. Um cliente com requisitos modestos e um orçamento correspondente poderia começar com um Modelo 30 relativamente barato. Mais tarde, se as necessidades do cliente aumentassem, seria possível atualizar para uma máquina mais rápida com mais memória, sem sacrificar o investimento no software já desenvolvido. As características de uma família são as seguintes:

- **Conjunto de instruções semelhante ou idêntico:** em muitos casos, exatamente o mesmo conjunto de instruções de máquina é aceito em todos os membros da família. Assim, um programa que executa em uma máquina também será executado em qualquer outra. Em alguns casos, o extremo inferior da família tem um conjunto de instruções que é um subconjunto daquele do topo da família. Isso significa que os programas podem subir, mas não descer.

⁶ O termo *mainframe* é usado para os computadores maiores e mais poderosos, que não sejam supercomputadores. As características típicas de um mainframe são que ele admite um grande banco de dados, tem hardware de E/S elaborado e é usado como recurso de processamento de dados central.

Tabela 2.4 Principais características da família System/360

Característica	Modelo 30	Modelo 40	Modelo 50	Modelo 65	Modelo 75
Tamanho máximo da memória (bytes)	64K	256K	256K	512K	512K
Taxa de dados da memória (MBytes/seg)	0,5	0,8	2,0	8,0	16,0
Tempo do ciclo do processador (μ s)	1,0	0,625	0,5	0,25	0,2
Velocidade relativa	1	3,5	10	21	50
Número máximo de canais de dados	3	3	4	6	6
Taxa de dados máxima em um canal (Kbytes/s)	250	400	800	1 250	1 250

- **Sistema operacional semelhante ou idêntico:** o mesmo sistema operacional básico está disponível para todos os membros da família. Em alguns casos, recursos adicionais são acrescentados aos membros de mais alto nível.
- **Velocidade aumentada:** a taxa de execução de instruções aumenta, dos membros mais baixos aos mais altos da família.
- **Número cada vez maior de portas de E/S:** o número de portas de E/S aumenta, dos membros mais baixos aos mais altos da família.
- **Tamanho de memória crescente:** o tamanho da memória principal aumenta, dos membros mais baixos aos mais altos da família.
- **Maior custo:** em determinado ponto no tempo, o custo de um sistema aumenta dos membros mais baixos aos mais altos da família.

Como esse conceito de família poderia ser implementado? As diferenças foram conseguidas com base em três fatores: velocidade básica, tamanho e grau de simultaneidade (Stevens, 1964⁴). Por exemplo, velocidade maior na execução de determinada instrução poderia ser obtida pelo uso de um circuito mais complexo na ALU, permitindo que as suboperações sejam executadas em paralelo. Outra forma de aumentar a velocidade era aumentar a largura do caminho de dados entre a memória principal e a CPU. No Modelo 30, somente 1 byte (8 bits) poderia ser apanhado da memória principal de cada vez, contra 8 bytes por vez no Modelo 75.

O System/360 não apenas ditou o curso futuro da IBM, mas também teve um impacto profundo sobre a indústria inteira. Muitos de seus recursos tornaram-se padrão de outros computadores grandes.

DEC PDP-8 No mesmo ano que a IBM entregou o primeiro System/360, ocorreu outra primeira remessa importantíssima: o PDP-8, da *Digital Equipment Corporation* (DEC). Na época em que o computador padrão exigia uma sala com ar condicionado, o PDP-8 (apelidado de minicomputador pelo setor, em decorrência da minissaia da época) era pequeno o suficiente para poder ser colocado sobre uma bancada de laboratório ou embutido em outro equipamento. Ele não podia fazer tudo o que o mainframe fazia, mas a US\$ 16 000, ele era barato o suficiente para que cada técnico de laboratório tivesse um. Ao contrário, a série de computadores mainframe System/360, introduzida apenas alguns meses antes, custava centenas de milhares de dólares.

O baixo custo e o pequeno tamanho do PDP-8 permitiram que outro fabricante adquirisse um PDP-8 e o integrasse a um sistema total para revenda. Esses outros fabricantes passaram a ser conhecidos como *original equipment manufacturers* (OEMs), e o mercado de OEM tornou-se e continua a ser um segmento importante do mercado de computadores.

O PDP-8 foi um sucesso imediato e fez a fortuna da DEC. Essa máquina e outros membros da família PDP-8 que a seguiram (Tabela 2.5) alcançaram um estado de produção reservados inicialmente para computadores IBM, com cerca de 50 000 máquinas vendidas nos doze anos seguintes. Conforme a história oficial da DEC relata, o PDP-8 “estabeleceu o conceito de minicomputadores, preparando o caminho para uma indústria multibilionária”. Ela também estabeleceu a DEC como o fornecedor de minicomputadores número um e, na época em que o PDP-8 alcançou o fim de sua vida útil, a DEC era o fabricante de computadores número dois, atrás somente da IBM.

Tabela 2.5 Evolução do PDP-8 (VOELKER, 1988^b)

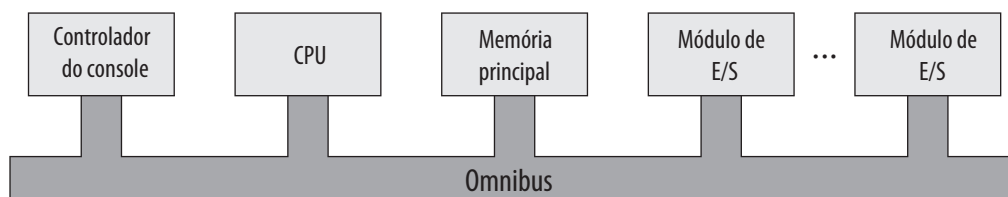
Modelo	Entregue inicialmente	Custo do processador + 4K palavras de memória de 12 bits (\$ 1000s)	Taxa de dados da memória (palavras/ μ s)	Volume (pés cúbicos)	Inovações e melhorias
PDP-8	4/65	16,2	1,26	8,0	Produção automática de <i>wire-wrapping</i>
PDP-8/5	9/66	8,79	0,08	3,2	Execução sequencial de instruções
PDP-8/1	4/68	11,6	1,34	8,0	Circuitos integrados em média escala
PDP-8/L	11/68	7,0	1,26	2,0	Gabinete menor
PDP-8/E	3/71	4,99	1,52	2,2	Omnibus
PDP-8/M	6/72	3,69	1,52	1,8	Gabinete com a metade do tamanho e com menos <i>slots</i> que o 8/E
PDP-8/A	1/75	2,6	1,34	1,2	Memória semicondutora; processador de ponto flutuante

Ao contrário da arquitetura comutada central (Figura 2.5) usada pela IBM em seus sistemas 700/7000 e 360, os modelos mais recentes do PDP-8 usavam uma estrutura que agora é praticamente universal para microcomputadores: a estrutura de barramento, ilustrada na Figura 2.9. O barramento do PDP-8, chamado Omnibus, consiste em 96 caminhos de sinal separados, usados para transportar sinais de controle, endereço e dados. Como todos os componentes do sistema compartilham um conjunto comum de caminhos de sinal, seu uso precisa ser controlado pela CPU. Essa arquitetura é altamente flexível, permitindo que os módulos sejam conectados ao barramento para criar várias configurações.



Gerações posteriores

Além da terceira geração, existe pouco consenso geral sobre a definição das gerações de computadores. A Tabela 2.2 sugere ter havido diversas gerações posteriores, com base nos avanços na tecnologia de circuito integrado. Com a introdução da integração em grande escala (LSI, do inglês *large-scale integration*), mais de 1 000 componentes podem ser colocados em um único chip de circuito integrado. A integração em escala muito grande (VLSI, do inglês *very-large-scale integration*) alcançou mais de 10 000 componentes por chip, enquanto os chips com integração em escala ultragrande (ULSI, do inglês *ultra-large-scale integration*) podem conter mais de um milhão de componentes.

Figura 2.9 Estrutura de barramento do PDP-8

Com o rápido ritmo da tecnologia, a alta taxa de introdução de novos produtos e a importância do software e das comunicações, além do hardware, a classificação por geração torna-se menos clara e menos significativa. Pode-se dizer que a aplicação comercial de novos desenvolvimentos resultou em uma mudança importante no início da década de 1970 e que os resultados dessas mudanças ainda estão sendo estudados. Nesta seção, mencionamos dois dos mais importantes desses resultados.

MEMÓRIA SEMICONDUTORA A primeira aplicação da tecnologia de circuito integrado aos computadores foi a construção do processador (a unidade de controle e a unidade lógica e aritmética) em chips de circuito integrado. Mas também descobriu-se que essa mesma tecnologia poderia ser usada para construir memórias.

Nas décadas de 1950 e 1960, a maior parte da memória do computador era construída a partir de pequenos anéis de material ferromagnético, cada um com cerca de 1/16 de polegada de diâmetro. Esses anéis eram montados em grades de pequenos fios suspensos em pequenas telas dentro do computador. Magnetizado em uma direção, um anel (chamado *core*) representava 1; magnetizado na outra direção, ele representava 0. A memória de núcleo magnético era relativamente rápida; era necessário apenas um milionésimo de segundo para ler um bit armazenado na memória. Mas ela era cara, volumosa e usava leitura destrutiva: o simples ato de ler um *core* apagava os dados armazenados nele. Portanto, era necessário instalar circuitos para restaurar os dados assim que eles fossem extraídos.

Então, em 1970, a Fairchild produziu a primeira memória de semicondutor relativamente grande. Esse chip, aproximadamente com o tamanho de um único *core*, poderia manter 256 bits de memória. Ela era não destrutiva e muito mais rápida que o *core*; levava apenas 70 bilionésimos de segundo para ler um bit. Porém, o custo por bit era mais alto do que o do *core*.

Em 1974, ocorreu um evento embrionário: o preço por bit da memória semicondutora caiu para menos que o preço por bit da memória *core*. Depois disso, houve um declínio contínuo e rápido no custo da memória acompanhado por um aumento correspondente na densidade da memória física. Isso preparou o caminho para máquinas menores e mais rápidas do que aquelas de alguns anos antes, com memórias maiores e mais caras. Os desenvolvimentos na tecnologia da memória, juntamente com os desenvolvimentos na tecnologia de processador — a serem discutidos em seguida —, mudaram a natureza dos computadores em menos de uma década. Embora os computadores volumosos e caros continuem sendo uma parte do panorama, o computador também foi levado ao “usuário final”, como máquinas de escritório e computadores pessoais.

Desde 1970, a memória semicondutora tem passado por 13 gerações: 1K, 4K, 16K, 64K, 256K, 1M, 4M, 16M, 64M, 256M, 1G, 4G, e, no momento em que este livro era escrito, 16 Gbits em um único chip ($1K = 2^{10}$, $1M = 2^{20}$, $1G = 2^{30}$). Cada geração forneceu quatro vezes a densidade de armazenamento da geração anterior, acompanhada pelo custo por bit e tempo de acesso em declínio.

MICROPROCESSADORES Assim como a densidade dos elementos nos chips de memória continuava a subir, a densidade dos elementos dos chips do processador subiram. Com o passar do tempo, mais e mais elementos eram colocados em cada chip, de modo que menos e menos chips eram necessários para construir um único processador do computador.

Uma descoberta inovadora foi alcançada em 1971, quando a Intel desenvolveu seu 4004. Ele foi o primeiro chip a conter em si *todos* os componentes de uma CPU: nascia o microprocessador.

O 4004 pode somar dois números de 4 bits e pode multiplicar apenas pela adição repetida. Pelos padrões de hoje, o 4004 é desesperadamente primitivo, mas ele marcou o início de uma evolução contínua da capacidade e do poder do microprocessador.

Essa evolução pode ser vista mais facilmente no número de bits com que o processador lida de cada vez. Não existe uma medida clara disso, mas talvez a melhor medida seja a largura do barramento: o número de bits de dados que podem ser trazidos ou enviados do processador de cada vez. Outra medida é o número de bits no acumulador ou no conjunto de registradores de uso geral. Normalmente, essas medidas coincidem, mas nem sempre. Por exemplo, diversos microprocessadores foram desenvolvidos para operarem sobre números de 16 bits nos registradores, mas só podem ler e escrever 8 bits de cada vez.

O próximo passo importante na evolução do microprocessador foi a introdução, em 1972, do Intel 8008. Esse foi o primeiro microprocessador de 8 bits e tinha quase o dobro da complexidade do 4004.

Nenhum desses passos teria o mesmo impacto do próximo evento importante: a introdução do Intel 8080 em 1974. Esse foi o primeiro microprocessador de uso geral. Enquanto o 4004 e o 8008 tinham sido projetados para aplicações específicas, o 8080 foi projetado para ser a CPU de um microcomputador de uso geral. Assim como o 8008, o 8080 é um microprocessador de 8 bits, porém, é mais rápido, tem um conjunto de instruções mais rico e uma grande capacidade de endereçamento.

Praticamente na mesma época, microprocessadores de 16 bits começaram a ser desenvolvidos. Porém, somente no final da década de 1970 é que apareceram os poderosos microprocessadores de 16 bits de uso geral. Um destes foi o 8086. O próximo passo nessa tendência ocorreu em 1981, quando a Bell Laboratórios e a Hewlett-Packard desenvolveram microprocessadores de 32 bits, de único chip. A Intel introduziu seu microprocessador de 32 bits, o 80386, em 1985 (Tabela 2.6).

Tabela 2.6 Evolução dos microprocessadores Intel

(a) Processadores da década de 1970

	4004	8008	8080	8086	8088
Introduzido	1971	1972	1974	1978	1979
Velocidades de clock	108 kHz	108 kHz	2 MHz	5 MHz, 8 MHz, 10 MHz	5 MHz, 8 MHz
Largura do barramento	4 bits	8 bits	8 bits	16 bits	8 bits
Número de transistores	2 300	3 500	6 000	29 000	29 000
Dimensão mínima da tecnologia de fabricação (μm)	10		6	3	6
Memória endereçável	640 bytes	16 KB	64 KB	1 MB	1 MB

(b) Processadores da década de 1980

	80286	386TM DX	386TM SX	486TM DX CPU
Introduzido	1982	1985	1988	1989
Velocidades de clock	6–12,5 MHz	16–33 MHz	16–33 MHz	25–50 MHz
Largura do barramento	16 bits	32 bits	16 bits	32 bits
Número de transistores	134.000	275.000	275.000	1,2 milhão
Dimensão mínima da tecnologia de fabricação (μm)	1,5	1	1	0,8–1
Memória endereçável	16 MB	4 GB	16 MB	4 GB
Memória virtual	1 GB	64 TB	64 TB	64 TB
Cache	–	–	–	8 kB

(c) Processadores da década de 1990

	486TM SX	Pentium	Pentium Pro	Pentium II
Introduzido	1991	1993	1995	1997
Velocidades de clock	16–33 MHz	60–166 MHz	150–200 MHz	200–300 MHz
Largura do barramento	32 bits	32 bits	64 bits	64 bits
Número de transistores	1,185 milhão	3,1 milhões	5,5 milhões	7,5 milhões
Dimensão mínima da tecnologia de fabricação (μm)	1	0,8	0,6	0,35
Memória endereçável	4 GB	4 GB	64 GB	64 GB
Memória virtual	64 TB	64 TB	64 TB	64 TB
Cache	8 kB	8 kB	512 kB L1 e 1 MB L2	512 kB L2

(Continua)

Tabela 2.6 Evolução dos microprocessadores Intel (*continuação*)**(d) Processadores recentes**

	Pentium III	Pentium 4	Core 2 Duo	Core 2 Quad
Introduzido	1999	2000	2006	2008
Velocidades de clock	450–660 MHz	1,3–1,8 GHz	1,06–1,2 GHz	3 GHz
Largura do barramento	64 bits	64 bits	64 bits	64 bits
Número de transistores	9,5 milhões	42 milhões	167 milhões	820 milhões
Dimensão mínima da tecnologia de fabricação (nm)	250	180	65	45
Memória endereçável	64 GB	64 GB	64 GB	64 GB
Memória virtual	64 TB	64 TB	64 TB	64 TB
Cache	512 KB L2	256 KB L2	2 MB L2	6 MB L2



2.2 Projetando visando ao desempenho

Ano após ano, o custo dos sistemas de computação continua a cair drasticamente, enquanto o desempenho e a capacidade desses sistemas continua a subir de forma igualmente drástica. Em uma loja de atacado local, você pode conseguir um computador pessoal por menos de US\$ 1 000, com desempenho melhor que o de um mainframe IBM de 10 anos atrás. Assim, temos praticamente potência de computação “grátis”. E essa revolução tecnológica contínua permitiu o desenvolvimento de aplicações de incrível complexidade e poder. Por exemplo, as aplicações de desktop que exigem a grande potência dos sistemas atuais baseados em microprocessador incluem

- processamento de imagens;
- reconhecimento de voz;
- videoconferência;
- criação de multimídia;
- anotação de arquivos de voz e vídeo; e
- modelagem de simulação.

Os sistemas de estação de trabalho agora admitem aplicações de engenharia e científicas altamente sofisticadas, além de sistemas de simulação, e têm a capacidade de suportar aplicações de imagem e vídeo. Além disso, as empresas estão contando com servidores cada vez mais poderosos para lidar com o processamento de transação e banco de dados, e para dar suporte a redes cliente/servidor massivas, que substituíram os imensos centros de computador mainframe do passado.

O que é fascinante sobre tudo isso, do ponto de vista da arquitetura e da organização do computador, é que, por um lado, os blocos de montagem básicos para os milagres do computador de hoje são praticamente os mesmos daqueles do computador IAS de 50 anos atrás, enquanto, por outro lado, as técnicas para espremer a última gota de desempenho dos materiais em mãos têm se tornado cada vez mais sofisticadas.

Essa observação serve como um princípio de orientação para a apresentação neste livro. Enquanto prosseguirmos pelos diversos elementos e componentes de um computador, dois objetivos são buscados. Primeiro, o livro explica a funcionalidade fundamental em cada área em consideração, e segundo, explora as técnicas exigidas para alcançar o máximo de desempenho. No restante desta seção, destacamos alguns dos fatores mais fortes para se alcançar o máximo de desempenho.



Velocidade do microprocessador

O que dá aos processadores Intel x86 ou computadores mainframe da IBM essa potência incrível é a busca implacável de velocidade pelos fabricantes de chip de processador. A evolução dessas máquinas continua a comprovar a lei de Moore, mencionada anteriormente. Como essa lei sustenta, os fabricantes de chips podem desencadear

uma nova geração de chips a cada três anos — com quatro vezes a quantidade de transistores. Em chips de memória, isso quadruplicou a capacidade da memória de acesso aleatório e dinâmico (DRAM, do inglês *dynamic random-access memory*), ainda a tecnologia básica para a memória principal, a cada três anos. Nos microprocessadores, a adição de novos circuitos, e o aumento de velocidade que vem da redução das distâncias entre eles, melhorou o desempenho de quatro a cinco vezes a cada três anos ou mais desde que a Intel lançou sua família x86 em 1978.

Mas a velocidade bruta do microprocessador não alcança seu potencial a menos que receba um fluxo constante de trabalho para fazer na forma de instruções de computador. Qualquer coisa que atrapalhe esse fluxo suave mina a potência do processador. Consequentemente, enquanto os fabricantes de chips estiverem ocupados aprendendo a fabricar chips com densidade cada vez maior, os projetistas de processadores deverão aparecer com técnicas ainda mais elaboradas para alimentar o monstro. Entre as técnicas embutidas nos processadores contemporâneos estão as seguintes:

- **Previsão de desvio:** o processador antecipa o código de instrução apanhado da memória e prevê quais desvios, ou grupos de instruções, provavelmente serão processados em seguida. Se o processador acertar na maior parte do tempo, ele poderá buscar antecipadamente as instruções corretas e mantê-las em um buffer, de modo que o processador continue ocupado. Os exemplos mais sofisticados dessa estratégia preveem não apenas o próximo desvio, mas múltiplos desvios adiante. Assim, a previsão de desvio aumenta a quantidade de trabalho disponível para o processador executar.
- **Análise de fluxo de dados:** o processador analisa quais instruções são dependentes dos resultados uma da outra, ou dos dados, para criar uma sequência otimizada de instruções. De fato, as instruções são escalonadas para serem executadas quanto estiverem prontas, independentemente da ordem original do programa. Isso impede atrasos desnecessários.
- **Execução especulativa:** usando a previsão de desvio e a análise do fluxo de dados, alguns processadores especulativamente executam instruções antes do seu surgimento real na execução do programa, mantendo os resultados em locais temporários. Isso permite que o processador mantenha seus mecanismos de execução o mais ocupados possível, executando instruções que provavelmente serão necessárias.

Estas e outras técnicas sofisticadas tornam-se necessárias pelo poder completo do processador. Elas fazem com que seja possível explorar a velocidade bruta do processador.



Balanco do desempenho

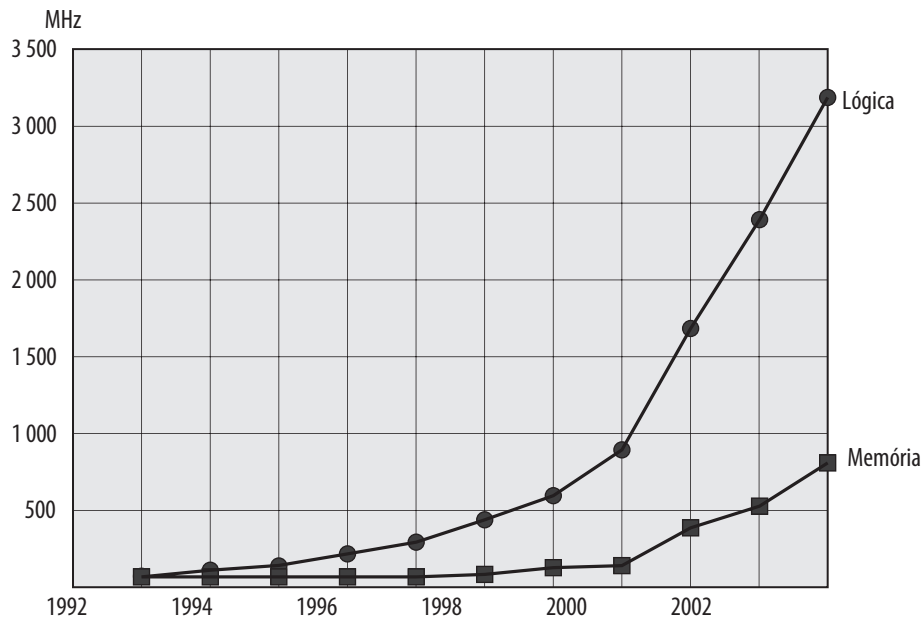
Embora a potência do processador tenha corrido na frente em velocidade espantosa, outros componentes críticos do computador não a acompanharam. O resultado é uma necessidade de procurar o equilíbrio do desempenho: um ajuste da organização e da arquitetura para compensar a diferença entre as capacidades dos diversos componentes.

Em nenhum outro lugar o problema criado por tais diferenças é mais crítico do que na interface entre o processador e a memória principal. Considere a história representada na Figura 2.10. Embora a velocidade do processador tenha aumentado rapidamente, a velocidade com que os dados podem ser transferidos entre a memória principal e o processador ficou para trás. A interface entre o processador e a memória principal é o caminho mais crítico no computador inteiro, pois é responsável por transportar um fluxo constante de instruções do programa e dados entre os chips de memória e o processador. Se a memória ou o caminho deixar de manter o ritmo com as demandas insistentes do processador, este entra em um estado de espera, e perde-se um tempo valioso de processamento.

Existem várias maneiras de um arquiteto de sistemas atacar esse problema, todas refletidas nos projetos contemporâneos de computador. Considere os seguintes exemplos:

- Aumentar o número de bits que são recuperados ao mesmo tempo, tornando a DRAM “mais larga” em vez de “mais profunda” e usando caminhos de dados largos no barramento.
- Alterar a interface da DRAM para torná-la mais eficiente, incluindo uma cache⁷ ou outro esquema de buffering no chip de DRAM.
- Reduzir a frequência de acesso à memória incorporando estruturas de cache cada vez mais complexas e eficientes entre o processador e a memória principal. Isso inclui a incorporação de uma ou mais caches no chip do processador, bem como uma cache, fora do chip, próxima do chip do processador.

⁷ Uma cache é uma memória rápida, relativamente pequena, interposta entre uma memória maior e mais lenta e a lógica que acessa a memória maior. A cache mantém dados acessados recentemente e é projetada para agilizar o acesso subsequente aos mesmos dados. As caches são discutidas no Capítulo 4.

Figura 2.10 Diferença de desempenho entre lógica e memória (Borkar, 2003)

- Aumentar a largura de banda de interconexão entre os processadores e a memória usando barramentos de velocidade mais alta e usando uma hierarquia de barramentos para armazenar e estruturar o fluxo de dados.

Outra área de foco de projeto é o tratamento dos dispositivos de E/S. À medida que os computadores se tornam mais rápidos e mais capazes, aplicações mais sofisticadas são desenvolvidas para dar suporte ao uso de periféricos com demandas intensivas de E/S. A Figura 2.11 oferece alguns exemplos de dispositivos periféricos típicos em uso nos computadores pessoais e estações de trabalho. Esses dispositivos criam demandas consideráveis de vazão de dados. Embora a geração atual de processadores possa tratar dos dados lançados por esses dispositivos, ainda resta o problema de movimentar esses dados entre o processador e o periférico. As estratégias aqui incluem esquemas de caching e buffering, mais o uso de barramentos de interconexão de maior velocidade e estruturas de barramentos mais elaboradas. Além disso, o uso de configurações de processador múltiplo pode auxiliar a satisfazer as demandas de E/S.

A chave em tudo isso é o equilíbrio. Os projetistas constantemente lutam para equilibrar as demandas de vazão e processamento dos componentes do processador, memória principal, dispositivos de E/S e estruturas de interconexão. Esse projeto precisa ser constantemente repensado para lidar com dois fatores em constante evolução:

- A taxa em que o desempenho está mudando nas diversas áreas da tecnologia (processador, barramentos, memória, periféricos) difere bastante de um tipo de elemento para outro.
- Novas aplicações e novos dispositivos periféricos constantemente mudam a natureza da demanda sobre o sistema em termos do perfil de instrução típico e dos padrões de acesso aos dados.

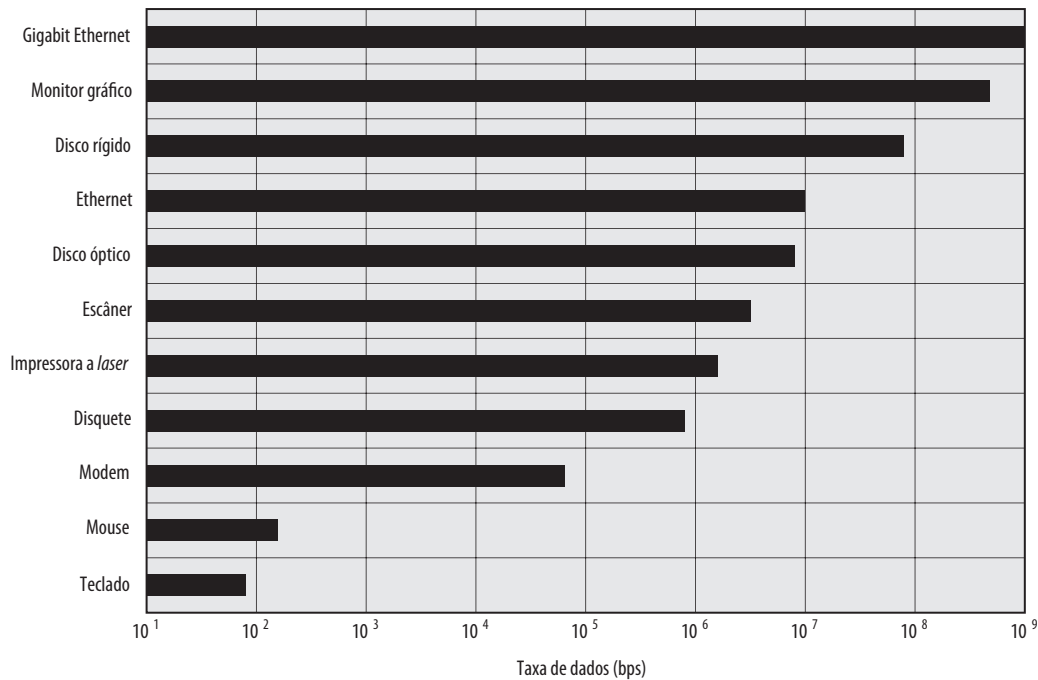
Assim, o projeto do computador é uma forma de arte em constante evolução. Este livro tenta apresentar os fundamentos nos quais essa forma de arte é baseada e apresentar uma análise do estado atual dela.



Melhorias na organização e na arquitetura do chip

À medida que os projetistas lutam com o desafio de balancear o desempenho do processador com o da memória principal e de outros componentes do computador, permanece a necessidade de aumentar a velocidade do processador. Para isso, existem três técnicas:

- Aumentar a velocidade de hardware do processador. Esse aumento deve-se fundamentalmente ao encolhimento do tamanho das portas lógicas no chip do processador, de modo que mais portas possam ser reunidas mais de perto, aumentando a taxa de clock. Com portas mais próximas, o tempo de propagação para os

Figura 2.11 Taxas de dados típicas dos dispositivos de E/S

sinais é significativamente reduzido, permitindo um aumento de velocidade do processador. Um aumento na taxa de clock significa que operações individuais são executadas mais rapidamente.

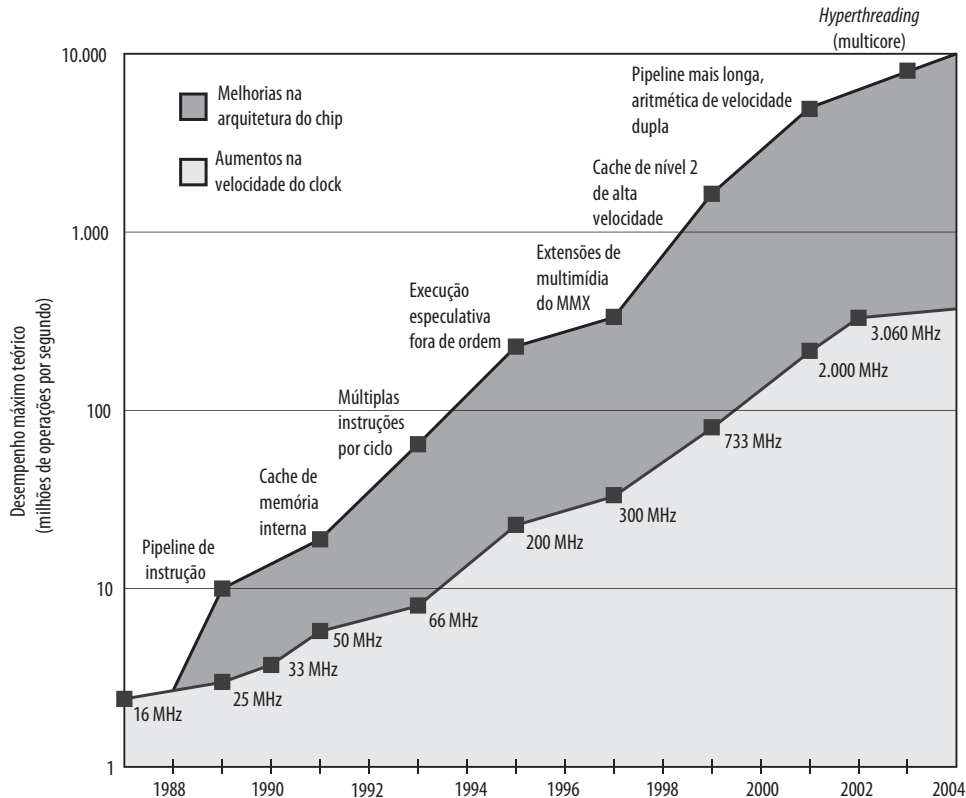
- Aumentar o tamanho e a velocidade das caches interpostas entre o processador e a memória principal. Em particular, dedicando uma parte do próprio chip do processador à cache, os tempos de acesso à cache caem significativamente.
- Fazer mudanças na organização e na arquitetura do processador, que aumentam a velocidade efetiva da execução da instrução. Tipicamente, isso envolve o uso do paralelismo de uma forma ou de outra.

Tradicionalmente, o fator dominante nos ganhos de desempenho tem sido em aumentos na velocidade do clock e densidade lógica. A Figura 2.12 ilustra essa tendência para chips de processador Intel. Porém, à medida que a velocidade do clock e a densidade lógica aumentam, diversos obstáculos se tornam mais significativos (Intel Research and Development, 2004^m):

- **Potência:** à medida que a densidade da lógica e a velocidade do clock em um chip aumentam, também aumenta a densidade de potência (Watts/cm²). A dificuldade de dissipar o calor gerado em chips de alta densidade e velocidade está se tornando um sério problema de projeto (Gibbs, 2004ⁿ; Borkar, 2003^e).
- **Atraso de RC:** a velocidade em que os elétrons podem fluir em um chip entre os transistores é limitada pela resistência (R) e capacitância (C) dos fios de metal que os conectam; especificamente, o atraso aumenta à medida que o produto RC aumenta. À medida que os componentes no chip diminuem de tamanho, as interconexões de fio se tornam mais finas, aumentando a resistência. Além disso, os fios estão mais próximos, aumentando a capacitância.
- **Latência da memória:** as velocidades de memória limitam as velocidades do processador, conforme já foi discutido.

Assim, haverá mais ênfase nas abordagens de organização e arquitetura para melhorar o desempenho. A Figura 2.12 destaca as principais mudanças feitas no decorrer dos anos para aumentar o paralelismo e, portanto, a eficiência computacional dos processadores. Essas técnicas são discutidas em outros capítulos do livro.

A partir do final da década de 1980, e continuando por cerca de 15 anos, duas estratégias principais têm sido usadas para aumentar o desempenho além do que pode ser alcançado simplesmente aumentando a velocidade

Figura 2.12 Desempenho do microprocessador Intel (Gibbs, 2004ⁿ)

do clock. Primeiro, tem havido um aumento na capacidade da cache. Agora, existem normalmente dois ou três níveis de cache entre o processador e a memória principal. À medida que a densidade do chip tem aumentado, mais da memória cache tem sido incorporada no chip, permitindo um acesso mais rápido a ela. Por exemplo, o chip Pentium original dedicava cerca de 10% da área do chip a uma cache. O chip do Pentium 4 mais recente dedica cerca de metade de sua área às caches.

Segundo, a lógica de execução de instrução dentro de um processador tornou-se cada vez mais complexa para permitir a execução paralela das instruções dentro do processador. Duas técnicas de projeto dignas de nota são pipeline e superescalar. Um pipeline funciona como uma linha de montagem em uma fábrica, permitindo que diferentes estágios de execução de diferentes instruções ocorram ao mesmo tempo pelo pipeline. Uma técnica superescalar basicamente permite múltiplos pipelines dentro de um único processador, de modo que as instruções que não dependem umas das outras possam ser executadas em paralelo.

Essas duas técnicas estão alcançando um ponto sem retorno. A organização interna dos processadores contemporâneos é excessivamente complexa e capaz de comprimir muito paralelismo do fluxo de instruções. Parece provável que aumentos mais significativos nessa direção serão relativamente modestos (Gibbs, 2004ⁿ). Com três níveis de cache no chip do processador, cada um oferecendo capacidade substancial, também parece que os benefícios da cache estejam chegando a um limite.

Porém, simplesmente contar com o aumento na taxa de clock para aumentar o desempenho faz com que nos deparemos com o problema de dissipação de potência já citado. Quanto maior a taxa de clock, maior a quantidade de potência a ser dissipada, sem falar que alguns limites físicos fundamentais já estão sendo atingidos.

Com todas essas dificuldades em mente, os projetistas passaram para uma técnica fundamentalmente nova para melhorar o desempenho: colocar múltiplos processadores no mesmo chip, com uma grande cache compartilhada. O uso de múltiplos processadores no mesmo chip, também conhecido como múltiplos cores, ou **multicore**, oferece o potencial de aumentar o desempenho sem aumentar a taxa de clock. Estudos indicam que, dentro de um processador, o aumento no desempenho é aproximadamente proporcional à raiz quadrada do aumento na

complexidade (Borkar, 2003). Mas, se o software puder suportar o uso efetivo de múltiplos processadores e, então, dobrar o número de processadores, quase dobra também seu desempenho. Assim, a estratégia é usar dois processadores mais simples no chip, ao invés de um mais complexo.

Além disso, com dois processadores, caches maiores são justificadas. Isso é importante porque o consumo de potência da lógica da memória em um chip é muito menor do que o da lógica de processamento. Nos próximos anos, podemos esperar que a maior parte dos novos chips de processador tenha múltiplos processadores.



2.3 Evolução da arquitetura Intel x86

No decorrer deste livro, contamos com muitos exemplos concretos de projeto e implementação de computador para ilustrar os conceitos e esclarecer as escolhas. Quase sempre, o livro conta com exemplos de duas famílias de computadores: o Intel X86 e a arquitetura ARM. As ofertas de x86 atuais representam os resultados de décadas de esforço de projeto em computadores com conjunto complexo de instruções (CISC). O x86 atual incorpora os sofisticados princípios de projeto antigamente encontrados apenas em mainframes e supercomputadores e serve como um excelente exemplo de projeto CISC. Uma técnica alternativa de projeto de processador é o computador com conjunto reduzido de instruções (RISC). A arquitetura ARM é usada em uma grande variedade de sistemas embarcados e é um dos sistemas baseados em RISC mais poderosos e bem projetados no mercado.

Nesta seção e na seguinte, oferecemos uma rápida visão geral desses dois sistemas.

Em termos de fatia de mercado, a Intel é considerada, a décadas, o fabricante número um de microprocessadores para sistemas não embarcados, uma posição da qual parece improvável recuar. A evolução de seu principal produto microprocessador serve como um bom indicador da tecnologia de computador em geral.

A Tabela 2.6 mostra essa evolução. É interessante que, à medida que os microprocessadores se tornaram mais rápidos e muito mais complexos, a Intel realmente acelerou o ritmo. A Intel costumava desenvolver microprocessadores, um após o outro, a cada quatro anos, mas agora espera manter os concorrentes acuados, retirando um ou dois anos desse tempo de desenvolvimento, como tem feito com a maioria das gerações recentes do x86.

Vale a pena listar alguns dos destaques da evolução da linha de produtos da Intel:

- **8080:** o primeiro microprocessador de uso geral do mundo. Esta era uma máquina de 8 bits, com um caminho de dados de 8 bits para a memória. O 8080 foi usado no primeiro computador pessoal, o Altair.
- **8086:** uma máquina muito mais poderosa, de 16 bits. Além de um caminho de dados mais largo e registradores maiores, o 8086 ostentava uma cache de instruções, ou fila, que fazia a pré-busca de algumas instruções antes que fossem executadas. Uma variante desse processador, o 8088, foi usado no primeiro computador pessoal da IBM, assegurando o sucesso da Intel. O 8086 é o primeiro aparecimento da arquitetura x86.
- **80286:** esta extensão do 8086 permitia o endereçamento de uma memória de 16 MB, em vez de apenas 1 MB.
- **80386:** a primeira máquina de 32 bits da Intel e uma reformulação geral do produto. Com uma arquitetura de 32 bits, o 80386 competia em complexidade e potência com os minicomputadores e mainframes introduzidos alguns anos antes. Esse foi o primeiro processador da Intel a aceitar multitarefa, significando que poderia executar vários programas ao mesmo tempo.
- **80486:** o 80486 introduziu o uso de tecnologia de cache muito mais sofisticada e poderosa, e pipeline sofisticado de instrução. O 80486 também ofereceu um coprocessador matemático embutido, tirando da CPU principal operações matemáticas complexas.
- **Pentium:** com o Pentium, a Intel introduziu o uso de técnicas superescalares, que permitem que múltiplas instruções sejam executadas em paralelo.
- **Pentium Pro:** o Pentium Pro continuou o movimento em direção à organização superescalar, iniciada com o Pentium, com o uso agressivo de renomeação de registrador, previsão de desvio, análise de fluxo de dados e execução especulativa.
- **Pentium II:** o Pentium II incorporou a tecnologia MMX da Intel, que foi projetada especificamente para processar dados de vídeo, áudio e gráfico de forma eficiente.
- **Pentium III:** o Pentium III incorpora instruções adicionais de ponto flutuante para dar suporte ao software gráfico 3D.

- **Pentium 4:** o Pentium 4 inclui ponto flutuante adicional e outras melhorias para multimídia.⁸
- **Core:** esse é o primeiro microprocessador Intel x86 com um dual core, referindo-se à implementação de dois processadores em um único chip.
- **Core 2:** o Core 2 estende a arquitetura para 64 bits. O Core 2 Quad oferece quatro processadores em um único chip.

Mais de 30 anos após sua introdução em 1978, a arquitetura x86 continua a dominar o mercado de processadores fora dos sistemas embarcados. Embora a organização e a tecnologia das máquinas x86 tenha mudado drasticamente durante as décadas, a arquitetura do conjunto de instruções evoluiu para permanecer compatível com versões anteriores. Assim, qualquer programa escrito em uma versão mais antiga da arquitetura x86 pode ser executado nas versões mais novas. Todas as mudanças na arquitetura do conjunto de instruções envolveram acréscimos ao conjunto de instruções, sem subtrações. A taxa de mudança tem sido o acréscimo de aproximadamente uma instrução por mês acrescentada à arquitetura durante os 30 anos (Anthes, 2008⁹), de modo que existem agora mais de 500 itens no conjunto de instruções.

O x86 oferece uma excelente ilustração dos avanços em hardware de computador durante os últimos 30 anos. O 8086 de 1978 foi introduzido com uma velocidade de clock de 5 MHz e tinha 29 000 transistores. Um Intel Core 2 Quad, introduzido em 2008, opera a 3 GHz, um ganho de velocidade com um fator de 600, e tem 820 milhões de transistores, cerca de 28 000 vezes a quantidade do 8086. Ainda assim, o Core 2 tem um invólucro ligeiramente maior que o 8086 e tem um custo comparável.



2.4 Sistemas embarcados e o ARM

A arquitetura ARM refere-se a uma arquitetura de processador que evoluiu dos princípios de projeto RISC e é usada em sistemas embarcados. O Capítulo 13 examina os princípios de projeto RISC com detalhes. Nesta seção, oferecemos uma breve visão geral do conceito de sistemas, e depois examinamos a evolução da ARM.



Sistemas embarcados

O termo *sistema embarcado* refere-se ao uso de eletrônica e software dentro de um produto, ao contrário de um computador de uso geral, como um sistema de laptop ou desktop. A seguir, veja uma boa definição geral:⁹

Sistema embarcado. Uma combinação de hardware e software de computador, e talvez partes adicionais mecânicas e outras, projetada para realizar uma função dedicada. Em muitos casos, os sistemas embarcados fazem parte de um sistema ou produto maior, assim como no caso de um sistema de freios ABS em um carro.

Os sistemas embarcados existem em muito mais quantidade do que os sistemas de computador de uso geral, abrangendo uma ampla gama de aplicações (Tabela 2.7). Esses sistemas possuem requisitos e restrições bastante variáveis, como os seguintes (Grimheden e Torngren, 2005⁹):

- Sistemas de pequenos a grandes, implicando restrições de custo muito diferentes e, portanto, diferentes necessidades de otimização e reuso.
- Relaxados para requisitos muito estritos e combinações de diferentes requisitos de qualidade, por exemplo, com relação a segurança, confiabilidade, tempo real, flexibilidade e legislação.
- Tempos de vida de curto a longo.
- Diferentes condições ambientais em termos de, por exemplo, radiação, vibrações e umidade.
- Diferentes características de aplicação, resultando em cargas estáticas *versus* dinâmicas, velocidade de lenta a rápida, tarefas com uso intenso de computação contra interface e/ou combinações destes.

⁸ Com o Pentium 4, a Intel passou de números romanos para números arábicos nos números de modelo.

⁹ Michael Barr. *Embedded Systems Glossary*. Netrino Technical Library. Disponível em <<http://www.netrino.com/Publications/Glossary/index.php>>.

Tabela 2.7 Exemplos de sistemas embarcados e seus mercados (Noergarrd, 2005^p)

Mercado	Dispositivo embutido
Automotivo	Sistema de ignição Controle de motor Sistema de freio
Eletrônico (para consumo)	Televisores digitais e analógicos Caixas set-top (DVD, VCR, cabo) <i>Personal Digital Assistants</i> (PDA) Aparelhos de cozinha (refrigeradores, torradeiras, fornos de micro-ondas) Automóveis Brinquedos/jogos Telefones/celulares/pagers Câmeras Sistemas de posicionamento global (GPS, do inglês <i>global positioning systems</i>)
Controle industrial	Robótica e sistemas de controle para manufatura Sensores
Médico	Bombas de infusão Máquinas de diálise Dispositivos protéticos Monitores cardíacos
Automação de escritório	Máquinas de fax Fotocopiadoras Impressoras Monitores Escâneres

- Diferentes modelos de computação, variando desde sistemas de evento discreto até aqueles envolvendo dinâmica de tempo contínuo (normalmente conhecidos como sistemas híbridos).

Normalmente, os sistemas embarcados estão fortemente acoplados ao seu ambiente. Isso pode ocasionar restrições em tempo real impostas pela necessidade de interagir com o ambiente. Restrições, como velocidades de movimento exigidas, precisão de medição e durações de tempo exigidas, ditam a temporização das operações de software. Se múltiplas atividades tiverem que ser gerenciadas simultaneamente, tem-se restrições de tempo real mais complexas.

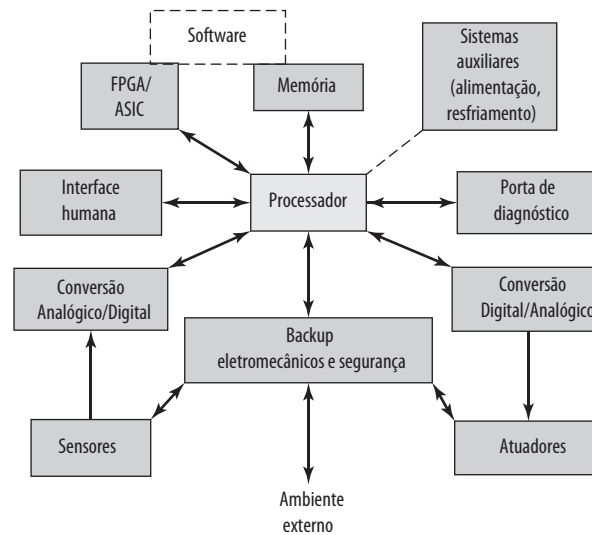
A Figura 2.13, baseada em Koopman (1996¹), mostra em termos gerais uma organização de sistema embarcado. Além do processador e da memória, existem diversos elementos que diferem do desktop ou laptop típico:

- Pode haver uma variedade de interfaces que permitem que o sistema meça, manipule e interaja de outras maneiras com o ambiente externo.
- A interface humana pode ser tão simples quanto uma luz piscando ou tão complicada quanto a visão robótica em tempo real.
- A porta de diagnóstico pode ser usada para diagnosticar o sistema que está sendo controlado, e não apenas para diagnóstico do computador.
- Hardware programável (FPGA), para aplicação específica (ASIC) ou mesmo um não digital pode ser utilizado para aumentar o desempenho ou a segurança.
- O software normalmente tem uma função fixa e é específico à aplicação.



Evolução do ARM

ARM é uma família de microprocessadores e microcontroladores baseados em RISC, projetados pela ARM Inc., Cambridge, Inglaterra. A empresa não fabrica processadores, mas projeta arquiteturas de microprocessador

Figura 2.13 Organização possível de um sistema embarcado

e multicore, e as licencia aos fabricantes. Os chips ARM são processadores de alta velocidade, conhecidos por seu pequeno tamanho do *die* e baixos requisitos de potência. Eles são bastante utilizados em PDAs e outros dispositivos portáteis, incluindo jogos e telefones, além de uma grande variedade de produtos para o consumo. Chips ARM são os processadores presentes nos populares dispositivos iPod e iPhone da Apple. A ARM provavelmente é a arquitetura de processador embutido mais utilizada e certamente a arquitetura de processador mais utilizada de qualquer tipo no mundo.

As origens da tecnologia ARM vêm da empresa britânica Acorn Computers. No início da década de 1980, a Acorn ganhou um contrato da British Broadcasting Corporation (BBC) para desenvolver uma nova arquitetura de microcomputador para o BBC Computer Literacy Project. O sucesso desse contrato permitiu à Acorn prosseguir e desenvolver o primeiro processador RISC comercial, o Acorn RISC Machine (ARM). A primeira versão, ARM1, começou a operar em 1985 e foi usada para pesquisa e desenvolvimento interno, além de ser usada como um coprocessador na máquina da BBC. Também em 1985, a Acorn lançou o ARM2, que tinha maior funcionalidade e velocidade dentro do mesmo espaço físico. Outras melhorias foram alcançadas com o lançamento do ARM3 em 1989.

Durante esse período, a Acorn usou a empresa VLSI Technology para fazer a fabricação real dos chips de processador. A VLSI era licenciada para comercializar o chip por conta própria e teve algum sucesso fazendo com que outras empresas usassem o ARM em seus produtos, particularmente como um processador embutido.

O projeto ARM combinou com uma necessidade comercial crescente por um processador de alto desempenho, baixo consumo de energia, pequeno tamanho e baixo custo para aplicações embarcadas. Mas o desenvolvimento além disso estava fora do escopo das capacidades da Acorn. Consequentemente, uma nova empresa foi organizada, com Acorn, VLSI e Apple Computer como parceiros fundadores, conhecida como ARM Ltd. A Acorn RISC Machine tornou-se a Advanced RISC Machine.¹⁰ A primeira oferta da nova empresa, uma melhoria sobre o ARM3, foi designada como ARM6. Subsequentemente, a empresa introduziu diversas novas famílias, com maior funcionalidade e desempenho. A Tabela 2.8 mostra algumas características delas. Os números nessa tabela são apenas aproximações; os valores reais variam bastante para diferentes implementações.

De acordo com o site Web do ARM (arm.com), os processadores ARM são projetados para atender às necessidades de três categorias de sistemas:

¹⁰ A empresa retirou a designação *Advanced RISC Machine* no final da década de 1990. Agora, a arquitetura é conhecida simplesmente como ARM.

Tabela 2.8 Evolução da ARM

Família	Recursos notáveis	Cache	MIPS típico @ MHz
ARM1	RISC 32 bits	Nenhuma	
ARM2	Instruções de multiplicação e swap; unidade de gerenciamento de memória integrada, processador gráfico e de E/S	Nenhuma	7 MIPS @ 12 MHz
ARM3	Primeira a usar cache de processador	4 KB unificada	12 MIPS @ 25 MHz
ARM6	Primeira a aceitar endereços de 32 bits: unidade de ponto flutuante	4 KB unificada	28 MIPS @ 33 MHz
ARM7	SoC integrado	8 KB unificada	60 MIPS @ 60 MHz
ARM8	Pipeline de 5 estágios; previsão estática de desvio	8 KB unificada	84 MIPS @ 72 MHz
ARM9		16 KB/16 KB	300 MIPS @ 300 MHz
ARM9E	Instruções DSP melhoradas	16 KB/16 KB	220 MIPS @ 200 MHz
ARM10E	Pipeline de 6 estágios	32 KB/32 KB	
ARM11	Pipeline de 9 estágios	Variável	740 MIPS @ 665 MHz
Cortex	Pipeline superescalar de 13 estágios	Variável	2 000 MIPS @ 1 GHz
XScale	Processador de aplicações; pipeline de 7 estágios	32 KB/32 KB L1 512KB L2	1 000 MIPS @ 1,25 GHz

DSP = processador de sinal digital (do inglês *digital signal processor*)

SoC = sistema em um chip (do inglês *system on a chip*)

- **Sistemas embarcados de tempo real:** sistemas para aplicações de armazenamento, automotivas, industriais e de redes.
- **Plataformas de aplicação:** dispositivos executando sistemas operacionais abertos, incluindo Linux, Palm OS, Symbian OS e Windows CE em aplicações sem fio, entretenimento e imagens digitais.
- **Aplicações seguras:** smart cards, placas SIM e terminais de pagamento.



2.5 Avaliação de desempenho

Na avaliação do hardware do processador e na definição de requisitos para novos sistemas, o desempenho é um dos principais parâmetros a se considerar, juntamente com custo, tamanho, segurança, confiabilidade e, em alguns casos, consumo de potência.

É difícil fazer comparações de desempenho significativas entre diferentes processadores, mesmo entre os processadores na mesma família. A velocidade bruta é muito menos importante do que como um processador funciona quando executa determinada aplicação. Infelizmente, o desempenho da aplicação depende não apenas da velocidade bruta do processador, mas do conjunto de instruções, da escolha da linguagem de implementação, da eficiência do compilador e da habilidade da programação feita para implementar a aplicação.

Começamos esta seção com uma visão de algumas medidas tradicionais de velocidade do processador. Depois, examinamos o enfoque mais comum para avaliar o desempenho do processador e do sistema de computação. Depois disso, veremos uma discussão de como avaliar os resultados de múltiplos testes. Finalmente, examinamos as observações produzidas considerando a lei de Amdahl.



Velocidade do clock e instruções por segundo

O CLOCK DO SISTEMA As operações realizadas por um processador, como busca e decodificação de uma instrução, realização de uma operação aritmética e assim por diante, são controladas por um clock do sistema. Normalmente, todas as operações começam com o pulso do clock. Assim, no nível mais fundamental, a velocidade de um processador é ditada pela frequência de pulso produzida pelo clock, medida em ciclos por segundo, ou Hertz (Hz).

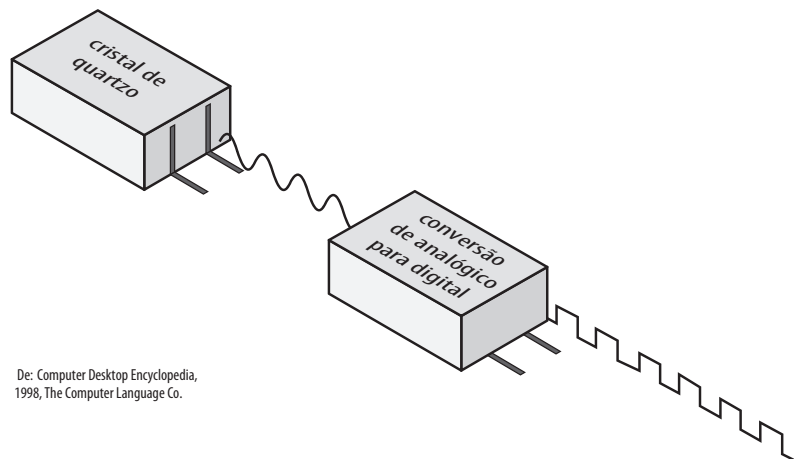
Normalmente, os sinais de clock são gerados por um cristal de quartzo, que gera uma onda de sinal constante enquanto a alimentação é aplicada. Essa onda é convertida em um *stream* de pulsos de voltagem digital, que é fornecido em um fluxo constante aos circuitos do processador (Figura 2.14). Por exemplo, um processador de 1 GHz recebe 1 bilhão de pulsos por segundo. A taxa de pulsos é conhecida como **taxa de clock**, ou **velocidade de clock**. Um incremento (ou pulso) do clock é conhecido como um **ciclo de clock**, ou um **clock tick**. O tempo entre os pulsos é o **tempo de ciclo**.

A taxa de clock não é arbitrária, mas precisa ser apropriada para o *layout* físico do processador. As ações no processador exigem que os sinais sejam enviados de um elemento do processador para outro. Quando um sinal é colocado em uma linha dentro do processador, é preciso alguma quantidade finita de tempo para os níveis de voltagem se estabilizarem, de modo que um valor preciso (1 ou 0) esteja disponível. Além do mais, dependendo do *layout* físico dos circuitos do processador, alguns sinais podem mudar mais rapidamente do que outros. Assim, as operações precisam ser sincronizadas e ritmadas de modo que valores de sinal elétrico (voltagem) apropriados estejam disponíveis para cada operação.

A execução de uma instrução envolve uma série de etapas discretas, como buscar a instrução na memória, decodificar as diversas partes da instrução, carregar e armazenar dados e realizar operações aritméticas e lógicas. Assim, grande parte das instruções na maioria dos processadores requer múltiplos ciclos de clock para completar. Algumas instruções podem usar apenas alguns ciclos, enquanto outras exigem dezenas. Além disso, quando é usado o pipeline, múltiplas instruções estão sendo executadas simultaneamente. Assim, uma comparação direta de velocidades de clock em diferentes processadores não diz a história toda sobre o desempenho.

TAXA DE EXECUÇÃO DE INSTRUÇÃO Um processador é controlado por um clock com uma frequência constante f ou, de modo equivalente, um tempo de ciclo constante τ , onde $\tau = 1/f$. Defina a contagem de instruções, I_p , para um programa como o número de instruções de máquina executadas para esse programa até que ele rode até o fim ou por algum intervalo de tempo definido. Observe que esse é o número de execuções de instrução e não o número de instruções no código objeto do programa. Um parâmetro importante é a média de ciclos por instrução (CPI, do inglês *cycles per instruction*) para um programa. Se todas as instruções exigissem o mesmo

Figura 2.14 Clock do sistema



De: Computer Desktop Encyclopedia,
1998, The Computer Language Co.

número de ciclos de clock, então o CPI seria um valor constante para um processador. Porém, em determinado processador, o número de ciclos de clock exigido varia para diferentes tipos de instruções, como load, store, branch e assim por diante. Considere que CPI_i seja o número de ciclos exigidos para a instrução tipo i , e I_i seja o número de instruções executadas de tipo i para determinado programa. Então, podemos calcular um CPI geral como a seguir:

$$CPI = \frac{\sum_{i=1}^n (CPI_i \times I_i)}{I_c} . \quad (2.1)$$

O tempo de processador T necessário para executar determinado programa pode ser expresso como:

$$T = I_c \times CPI \times \tau .$$

Podemos refinar essa formulação reconhecendo que, durante a execução de uma instrução, parte do trabalho é feito pelo processador, e parte do tempo uma palavra está sendo transferida da e para a memória. Nesse último caso, o tempo para transferir depende do tempo de ciclo da memória, que pode ser maior que o tempo de ciclo do processador. Podemos reescrever a equação anterior como:

$$T = I_c \times [p + (m \times k)] \times \tau ,$$

onde p é o número de ciclos de processador necessários para decodificar e executar a instrução, m é o número de referências de memória necessárias e k é a razão entre o tempo de ciclo da memória e o tempo de ciclo do processador. Os cinco fatores de desempenho na equação anterior (I_c, p, m, k, τ) são influenciados por quatro atributos do sistema: o projeto do conjunto de instruções (conhecido como *arquitetura do conjunto de instruções*), a tecnologia do compilador (quão efetivo é o compilador para produzir um programa em linguagem de máquina eficiente, de um programa em linguagem de alto nível), a implementação do processador e a hierarquia da cache e da memória. A Tabela 2.9, baseada em Hwang (1993³), é uma matriz em que uma dimensão mostra os cinco fatores de desempenho e a outra dimensão mostra os quatro atributos do sistema. Um X em uma célula indica um atributo do sistema que afeta um fator de desempenho.

Uma medida comum do desempenho para um processador é a taxa em que as instruções são executadas, expressa como milhões de instruções por segundo (MIPS, do inglês *millions of instructions per second*), conhecida como **taxa MIPS**. Podemos expressar a taxa MIPS em termos da taxa de clock e do CPI da seguinte forma:

$$\text{Taxa MIPS} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6} . \quad (2.2)$$

Por exemplo, considere a execução de um programa que resulta na execução de 2 milhões de instruções em um processador de 400 MHz. O programa consiste em quatro tipos principais de instruções. A mistura de instruções

Tabela 2.9 Fatores de desempenho e atributos do sistema

	I_c	p	m	k	τ
Arquitetura do conjunto de instruções	X	X			
Tecnologia do compilador	X	X	X		
Implementação do processador		X			X
Hierarquia da cache e da memória				X	X

Tabela 2.10 Mistura de instruções e CPI

Tipo de instrução	CPI	Número de instruções (%)
Aritmética e lógica	1	60%
Load/store com acerto de cache	2	18%
Desvio	4	12%
Referência de memória com falha de cache	8	10%

e o CPI para cada tipo de instrução aparecem na Tabela 2.10, com base no resultado de um experimento de *trace* de programa.

O CPI médio quando o programa é executado em um uniprocessador com os resultados de *trace* mostrados é $CPI = 0,6 + (2 \times 0,18) + (4 \times 0,12) + (8 \times 0,1) = 2,24$. A taxa MIPS correspondente é $(400 \times 10^6) / (2,24 \times 10^6) \approx 178$.

Outra medida de desempenho comum lida apenas com instruções de ponto flutuante. Estas são comuns em muitas aplicações científicas e de jogos. O desempenho do ponto flutuante é expresso como milhões de operações de ponto flutuante por segundo (MFLOPS, do inglês *million of floating-point operations per second*), definido da seguinte forma:

$$\text{Taxa MFLOPS} = \frac{\text{Número de operações de ponto flutuante executadas em um programa}}{\text{Tempo de execução} \times 10^6}$$



Benchmarks

Medidas como MIPS e MFLOPS provaram ser inadequadas para avaliar o desempenho dos processadores. Devido a diferenças nos conjuntos de instruções, a taxa de execução de instrução não é um meio válido de comparar o desempenho de diferentes arquiteturas. Por exemplo, considere esta instrução em linguagem de alto nível:

```
A = B + C /* considere todas as quantidades na memória principal */
```

Com a arquitetura tradicional do conjunto de instruções, conhecida como CISC, essa instrução pode ser compilada em uma instrução de processador:

```
add    mem(B),  mem(C),  mem (A)
```

Em uma máquina RISC típica, a compilação se pareceria com o seguinte:

```
load   mem(B),   reg(1);
load   mem(C),   reg(2);
add     reg(1),   reg(2),   reg(3);
store  reg(3),   mem (A)
```

Devido à natureza da arquitetura RISC (discutida no Capítulo 13), ambas as máquinas podem executar a instrução original em linguagem de alto nível aproximadamente no mesmo tempo. Se este exemplo for representativo das duas máquinas, então, se a máquina CISC for classificada com 1 MIPS, a máquina RISC seria classificada com 4 MIPS. Mas ambas realizam a mesma quantidade de trabalho em linguagem de alto nível na mesma quantidade de tempo.

Além do mais, o desempenho de certo processador em determinado programa pode não ser útil para determinar como esse processador funcionará em um tipo de aplicação muito diferente. Consequentemente, a partir do final da década de 1980 e início da seguinte, o interesse industrial e acadêmico passou para a medição do desempenho dos sistemas usando um conjunto de programas de benchmark. O mesmo conjunto de programas pode ser executado em diferentes máquinas, com os tempos de execução comparados.

Weicker (1990³) lista as características desejadas de um programa de benchmark:

1. É escrito em uma linguagem de alto nível, tornando-o portátil entre diferentes máquinas.
2. Representa um tipo particular de estilo de programação, como programação de sistemas, programação numérica ou programação comercial.
3. Pode ser medido com facilidade.
4. Tem uma ampla distribuição.

BENCHMARKS SPEC A necessidade comum nas comunidades industrial, acadêmica e de pesquisa para medidas de desempenho de computador geralmente aceitas tem levado ao desenvolvimento de pacotes de benchmark padronizados. Um pacote de benchmark é uma coleção de programas, definidos em uma linguagem de alto nível, que, juntos, tentam oferecer um teste representativo de um computador em determinada área de aplicação ou de programação de sistema. A mais conhecida dessa coleção de pacotes de benchmark é definida e mantida pela *System Performance Evaluation Corporation* (SPEC), um consórcio da indústria. As medidas de desempenho SPEC são bastante usadas para fins de comparação e pesquisa.

O mais conhecido dos pacotes de benchmark da SPEC é o SPEC CPU2006. Esse é o pacote padrão da indústria para aplicações com uso intensivo do processador. Ou seja, o SPEC CPU2006 é apropriado para medir o desempenho de aplicações que gastam a maior parte de seu tempo realizando cálculos, do que E/S. O pacote CPU2006 é baseado em aplicações existentes que já foram portadas para uma grande variedade de plataformas pelos membros do setor SPEC. Ele consiste em 17 programas de ponto flutuante escritos em C, C++ e Fortran, e 12 programas de inteiros escritos em C e C++. O pacote contém mais de 3 milhões de linhas de código. Essa é a quinta geração de pacotes com uso intensivo de CPU da SPEC, substituindo o SPEC CPU2000, SPEC CPU95, SPEC CPU92 e SPEC CPU89 (Henning, 2007⁴).

Outros pacotes SPEC são os seguintes:

- **SPECjvm98:** voltado para avaliar o desempenho dos aspectos combinados de hardware e software da plataforma cliente de *Java Virtual Machine* (JVM).
- **SPECjbb2000 (Java Business Benchmark):** um benchmark para avaliar aplicações de comércio eletrônico baseadas em Java no lado do servidor.
- **SPECweb99:** avalia o desempenho dos servidores da World Wide Web (WWW).
- **SPECmail2001:** criado para medir o desempenho do sistema que atua como servidor de correio.

AVALIANDO RESULTADOS Para obter uma comparação confiável do desempenho de diversos computadores, é preferível executar uma série de programas de benchmark diferentes em cada máquina e depois avaliar os resultados. Por exemplo, se houver m diferentes programas de benchmark, então uma **média aritmética** simples pode ser calculada da seguinte forma:

$$R_A = \frac{1}{m} \sum_{i=1}^m R_i \quad (2.3)$$

onde R_i é a taxa de execução de instrução na linguagem de alto nível para o i -ésimo programa de benchmark.

Uma alternativa é usar a **média harmônica**:

$$R_H = \frac{m}{\sum_{i=1}^m \frac{1}{R_i}} \quad (2.4)$$

No final das contas, o usuário se preocupa com o tempo de execução de um sistema, e não com sua taxa de execução. Se usarmos a média aritmética das taxas de instrução de diversos programas de benchmark, obtemos um resultado que é proporcional à soma dos inversos dos tempos de execução. Mas isso não é inversamente proporcional à soma dos tempos de execução. Em outras palavras, a média aritmética da taxa de instrução não se relaciona claramente ao tempo de execução. Por outro lado, a taxa de instrução da média harmônica é o inverso do tempo médio de execução.

Benchmarks SPEC não se preocupam com as taxas de execução de instrução. Em vez disso, duas métricas fundamentais são interessantes: uma métrica de velocidade e uma métrica de taxa. A **métrica de velocidade** mede a capacidade de um computador completar uma única tarefa. O SPEC define um *runtime* básico para cada programa de

benchmark usando uma máquina de referência. Os resultados para um sistema em teste são relatados como a **razão** entre o tempo de execução de referência e o tempo de execução do sistema. A razão é calculada da seguinte forma:

$$r_i = \frac{T_{ref_i}}{T_{sut_i}}, \quad (2.5)$$

onde T_{ref_i} é o tempo de execução do programa de benchmark i no sistema de referência e T_{sut_i} é o tempo de execução do programa de benchmark i no sistema em teste.

Como um exemplo de cálculo e relatório, considere o Sun Blade 6250, que consiste em dois chips com quatro cores, ou processadores, por chip. Um dos benchmarks de inteiros SPEC CPU2006 é o 464.h264ref. Essa é uma implementação de referência do H.264/AVC (*advanced video coding*), o padrão de compactação de vídeo mais moderno. O sistema Sun executa esse programa em 934 segundos. A implementação de referência requer 22.135 segundos. A razão é calculada como: $22.136/934 = 23,7$.

Como o tempo para o sistema em teste está no denominador, quanto maior a razão, mais alta é a velocidade. Uma medida de desempenho geral para o sistema em teste é calculada tirando-se a média dos valores para as razões de todos os 12 benchmarks de inteiros. A SPEC especifica o uso de uma **média geométrica**, definida da seguinte forma:

$$r_G = \left(\prod_{i=1}^n r_i \right)^{1/n}, \quad (2.6)$$

onde r_i é a razão para o i -ésimo programa de benchmark. Para o Sun Blade 6250, as razões de velocidade de inteiros do SPEC foram relatadas como mostra a Tabela 2.11.

A métrica de velocidade é calculada apanhando-se a 12ª raiz do produto das razões:

$$(17,5 \times 14 \times 13,7 \times 17,6 \times 14,7 \times 18,6 \times 17 \times 31,3 \times 23,7 \times 9,23 \times 10,9 \times 14,7)^{1/12} = 18,5$$

A **métrica de taxa** mede a vazão ou taxa de uma máquina executando uma série de tarefas. Para a métrica de taxa, várias cópias dos benchmarks são executadas simultaneamente. Em geral, o número de cópias é igual ao número de processadores na máquina. Novamente, uma razão é usada para relatar os resultados, embora o cálculo seja mais complexo. A razão é calculada da seguinte forma:

$$r_i = \frac{N \times T_{ref_i}}{T_{sut_i}}, \quad (2.7)$$

onde T_{ref_i} é o tempo de execução de referência para o benchmark i , N é o número de cópias do programa que são executadas simultaneamente, e T_{sut_i} é o tempo decorrido desde o início da execução do programa em todos os N processadores do sistema em teste, até o término de todas as cópias do programa. Novamente, uma média geométrica é calculada para determinar a medida de desempenho geral.

Tabela 2.11 Razões de velocidades de inteiros do SPEC

Benchmark	Razão
400.perlbench	17,5
401.bzip2	14,0
403.gcc	13,7
429.mcf	17,6
445.gobmk	14,7
456.hmmer	18,6

Benchmark	Razão
458.sjeng	17,0
462.libquantum	31,3
464.h264ref	23,7
471.omnetpp	9,23
473.astar	10,9
483.xalancbmk	14,7

SPEC escolheu usar uma média geométrica, pois é o mais apropriado para números normalizados, como razões. Fleming e Wallace (1986^v) demonstram que a média geométrica tem a propriedade de manter relacionamentos de desempenho de forma coerente, independentemente do computador usado como base para normalização.



Lei de Amdahl

Ao considerar o desempenho do sistema, os projetistas de sistemas de computação procuraram maneiras de melhorar o desempenho aperfeiçoando a tecnologia ou mudando o projeto. Alguns exemplos incluem o uso de processadores paralelos, o uso de uma hierarquia de cache de memória e *speedup* no tempo de acesso da memória e na taxa de transferência de E/S devido às melhorias na tecnologia. Em todos esses casos, é importante observar que um *speedup* em um aspecto da tecnologia ou projeto não resulta em uma melhoria correspondente no desempenho. Essa limitação é expressa de forma sucinta pela lei de Amdahl.

A lei de Amdahl foi proposta inicialmente por Gene Amdahl (1967^w), e lida com o potencial *speedup* de um programa usando múltiplos processadores em comparação com um único processador. Considere um programa rodando em um único processador, de modo que uma fração $(1 - f)$ do tempo de execução envolva um código inerentemente serial e uma fração f envolva código infinitamente paralelizável sem *overhead* de escalonamento. Considere que T seja o tempo de execução total do programa usando um único processador. Então, o *speedup* usando um processador paralelo com N processadores, que explora totalmente a parte paralela do programa, é o seguinte:

$$\begin{aligned} \text{Speedup} &= \frac{\text{tempo para executar programa em um único processador}}{\text{tempo para executar programa em } N \text{ processadores paralelos}} \\ &= \frac{T(1 - f) + Tf}{T(1 - f) + \frac{Tf}{N}} = \frac{1}{(1 - f) + \frac{f}{N}} \end{aligned}$$

Podemos chegar a duas conclusões importantes:

1. Quando f é pequeno, o uso de processadores paralelos tem pouco efeito.
2. Quando N se aproxima do infinito, o *speedup* é limitado por $1/(1 - f)$, de modo que existem retornos decrescentes para o uso de mais processadores.

Essas conclusões são muito pessimistas, uma declaração proposta inicialmente em Gustafson (1988^x). Por exemplo, um servidor pode manter múltiplas *threads* ou múltiplas tarefas para lidar com múltiplos clientes e executar as *threads* ou tarefas em paralelo até o limite do número de processadores. Muitas aplicações de banco de dados envolvem cálculos sobre grandes quantidades de dados, que podem ser divididos em múltiplas tarefas em paralelo. Apesar disso, a lei de Amdahl ilustra os problemas enfrentados pela indústria no desenvolvimento de máquinas multicore com um número cada vez maior de processadores: o software que roda nessas máquinas precisa ser adaptado para um ambiente de execução altamente paralelo, para explorar o poder do processamento paralelo.

A lei de Amdahl pode ser generalizada para avaliar qualquer melhoria de projeto ou técnica em um sistema de computação. Considere qualquer melhoria a um recurso de um sistema que resulte em um *speedup*, o qual pode ser expresso como:

$$\text{Speedup} = \frac{\text{Desempenho após melhoria}}{\text{Desempenho antes da melhoria}} = \frac{\text{Tempo de execução antes da melhoria}}{\text{Tempo de execução após melhoria}} \quad (2.8)$$

Suponha que o recurso do sistema seja usado durante a execução de uma fração do tempo f , antes da melhoria, e que o *speedup* desse recurso após a melhoria seja SU_f . Então, o *speedup* geral do sistema é:

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{SU_f}}$$

Por exemplo, suponha que uma tarefa utilize muitas operações de ponto flutuante, com 40% do tempo sendo consumido por operações de ponto flutuante. Com um novo projeto de hardware, o módulo de ponto flutuante é agilizado por um fator de K . Então, o *speedup* geral é:

$$\text{Speedup} = \frac{1}{0,6 + \frac{0,4}{K}}.$$

Assim, independentemente de K , o *speedup* máximo é 1,67.



2.6 Leitura recomendada e sites Web

Uma descrição da série IBM 7000 pode ser encontrada em Bell e Newell (1971^e). Há uma boa explicação do IBM 360 em Siewiorek, Bell e Newell, (1982^y) e do PDP-8 e outras máquinas DEC em Bell, Mudge e McNamara, (1978^z). Esses três livros também contêm diversos exemplos detalhados de outros computadores no decorrer da história dessas máquinas até o início da década de 1980. Um livro mais recente, que inclui um excelente conjunto de estudos de caso de máquinas históricas, é Blaauw e Brooks (1997^{aa}). Uma boa história do microprocessador é Betker, Fernando e Whalen (1997^{bb}).

Olukotun et al. (2007^{cc}), Hammond, Noylay e Olukotun (1997^{dd}) e Sakai (2002^{ee}) discutem a motivação para múltiplos processadores em um único chip.

Brey2009^{ff} oferece um bom estudo da linha de microprocessadores da Intel. A própria documentação da Intel também é boa (Intel Corp; 2008^{gg}).

A documentação mais completa, disponível para a arquitetura ARM, é de Seal (2000^{hh}).¹¹ Furber (2000ⁱⁱ) é outra excelente fonte de informações. Smith (2008^{jj}) é uma comparação interessante das abordagens ARM e x86 para processadores embutidos em dispositivos móveis sem fio.

Para ver discussões interessantes da lei de Moore e suas consequências, consulte Hutcheson e Hutcheson (1996^{kk}), Schaller (1997^{ll}) e Bohr (1998^{mm}).

Henning (2006ⁿⁿ) oferece uma descrição detalhada de cada um dos benchmarks no CPU2006. Smith (1988^{oo}) discute os méritos relativos das médias aritmética, harmônica e geométrica.



Sites Web recomendados

Intel Developer's Page: página Web da Intel para desenvolvedores, oferece um ponto de partida para acessar informações sobre o Pentium. Também inclui o Intel Technology Journal.

ARM: página web da ARM Limited, desenvolvedora da arquitetura ARM. Inclui documentação técnica.

Standard Performance Evaluation Corporation: SPEC é uma organização bastante reconhecida no setor de computação por seu desenvolvimento de benchmarks padronizados para medir e comparar o desempenho de diferentes sistemas de computação.

Top500 Supercomputer Site: oferece uma breve descrição de arquitetura e organização dos produtos de supercomputador atuais, incluindo comparações.

Charles Babbage Institute: oferece links para diversos sites Web que lidam com a história dos computadores.

¹¹ Conhecido na comunidade ARM como o "ARM ARM".

Principais termos, perguntas de revisão e problemas

Principais termos

Acumulador (AC)	Ciclo de instrução	<i>Opcode</i>
Lei de Amdahl	Registrador de instrução (IR)	<i>Original equipment manufacturer</i> (OEM)
Unidade lógica e aritmética (ALU)	Conjunto de instruções	Unidade de controle do programa
Benchmark	Circuito integrado (CI)	Contador de programa (PC)
Chip	Memória principal	SPEC
Canal de dados	Registrador de endereço da memória (MAR)	Computador de programa armazenado
Sistema embarcado	Registrador de buffer de memória (MBR)	Compatibilidade
Ciclo de execução	Microprocessador	Máquina de von Neumann
Ciclo de busca	Multicore	Wafer
Entrada/saída (E/S)	Multiplexador	Palavra
Registrador de buffer de instrução (IBR)		

Perguntas de revisão

- 2.1 O que é um computador de programa armazenado?
- 2.2 Quais são os quatro componentes principais de qualquer computador de uso geral?
- 2.3 No nível de circuito integrado, quais são os três constituintes principais de um sistema de computação?
- 2.4 Explique a lei de Moore.
- 2.5 Liste e explique as principais características de uma família de computadores.
- 2.6 Qual é a principal característica que distingue um microprocessador?

Problemas

- 2.1 Considere que $\mathbf{A} = A(1), A(2), \dots, A(1\ 000)$ e $\mathbf{B} = B(1), B(2), \dots, B(1\ 000)$ sejam dois vetores (*arrays* unidimensionais) compostos de 1 000 números em cada um, que são somados para formar um *array* $\mathbf{C}$ tal que $C(I) = A(I) + B(I)$ para $I = 1, 2, \dots, 1\ 000$. Usando o conjunto de instruções do IAS, escreva um programa para esse problema. Ignore o fato de que o IAS foi projetado para ter apenas 1 000 palavras de armazenamento.
- 2.2 a. No IAS, como ficaria a instrução de código de máquina para carregar o conteúdo do endereço de memória 2?
b. Quantas viagens à memória a CPU precisa fazer para completar essa instrução durante o ciclo de instrução?
- 2.3 No IAS, descreva em português o processo que a CPU precisa assegurar para ler um valor da memória e escrever um valor na memória em termos do que é colocado em MAR, MBR, barramento de endereço, barramento de dados e barramento de controle.
- 2.4 Dado o conteúdo de memória do computador IAS, mostrado a seguir,

Endereço	Conteúdo
08A	010FA210FB
08B	010FA0F08D
08C	020FA210FB

mostre o código em linguagem de montagem para o programa, começando no endereço 08A. Explique o que esse programa faz.

- 2.5 Na Figura 2.3, indique a largura, em bits, de cada caminho de dados (por exemplo, entre AC e ALU).
- 2.6 No IBM 360 Modelos 65 e 75, os endereços são espalhados em duas unidades separadas da memória principal (por exemplo, todas as palavras de número par em uma unidade e todas as palavras de número ímpar em outra). Qual poderia ser a finalidade dessa técnica?
- 2.7 Com referência à Tabela 2.4, vemos que o desempenho relativo do IBM 360 Modelo 75 é 50 vezes o do 360 Modelo 30, embora o tempo de ciclo de instrução seja apenas 5 vezes mais rápido. Como você explica essa discrepância?

- 2.8** Enquanto analisa a loja de computadores de Billy Bob, você escuta um cliente perguntando a ele qual é o computador mais rápido na loja que ele possa comprar. Billy Bob responde: “Você está olhando para nossos Macintoshes. O Mac mais rápido que temos trabalha com uma velocidade de clock de 1,2 gigahertz. Se você realmente quer a máquina mais rápida, então deve comprar nosso Intel Pentium IV de 2,4 gigahertz em vez disso”. Billy Bob está certo? O que você diria para ajudar esse cliente?
- 2.9** O ENIAC era uma máquina decimal, onde um registrador era representado por um anel de 10 válvulas. A qualquer momento, somente uma válvula estava no estado ON, representando um dos 10 dígitos. Supondo que o ENIAC tivesse a capacidade de ter várias válvulas no estado ON e OFF simultaneamente, por que essa representação é “esbanjadora” e que faixa de valores inteiros poderíamos representar usando 10 válvulas?
- 2.10** Um programa de benchmark é executado em um processador a 40 MHz. O programa executado consiste em 100.000 execuções de instrução, com a seguinte mistura de instruções e quantidade de ciclos de clock:

Tipo de instrução	Quantidade de instruções	Ciclos por instrução
Aritmética de inteiros	45 000	1
Transferência de dados	32 000	2
Ponto flutuante	15 000	2
Transferência de controle	8 000	2

Determine o CPI efetivo, a taxa de MIPS e o tempo de execução para esse programa.

- 2.11** Considere duas máquinas diferentes, com dois conjuntos de instruções diferentes, ambos tendo uma taxa de clock de 200 MHz. As medições a seguir são registradas nas duas máquinas rodando determinado conjunto de programas de benchmark:

Tipo de instrução	Quantidade de instruções (milhões)	Ciclos por instrução
Máquina A		
Aritmética e lógica	8	1
Load e store	4	3
Desvio	2	4
Outros	4	3
Máquina B		
Aritmética e lógica	10	1
Load e store	8	2
Desvio	2	4
Outros	4	3

- a. Determine o CPI efetivo, a taxa MIPS e o tempo de execução para cada máquina.
- b. Comente os resultados.
- 2.12** Os primeiros exemplos de projeto CISC e RISC são o VAX 11/780 e o IBM RS/6000, respectivamente. Usando um programa de benchmark típico, o resultado são as seguintes características de máquina:

Processador	Frequência de clock	Desempenho	Tempo de CPU
VAX 11/780	5 MHz	1 MIPS	12 × segundos
IBM RS/6000	25 MHz	18 MIPS	× segundos

A coluna final mostra que o VAX exigia 12 vezes mais tempo que o IBM, medido em tempo de CPU.

- a. Qual é o tamanho relativo da quantidade de instruções do código de máquina para esse programa de benchmark rodando nas duas máquinas?
- b. Quais são os valores de *CPI* para as duas máquinas?
- 2.13** Quatro programas de benchmark são executados em três computadores com os seguintes resultados:

	Computador A	Computador B	Computador C
Programa 1	1	10	20
Programa 2	1 000	100	20
Programa 3	500	1 000	50
Programa 4	100	800	100

A tabela mostra o tempo de execução em segundos, com 100 000 000 instruções executadas em cada um dos quatro programas. Calcule os valores de MIPS para cada computador para cada programa. Depois, calcule as médias aritmética e harmônica considerando pesos iguais para os quatro programas, e classifique os computadores com base na média aritmética e a média harmônica.

- 2.14** A tabela a seguir, baseada em dados relatados na literatura (Heath, 1984^{pp}), mostra os tempos de execução, em segundos, para cinco diferentes programas de benchmark em três máquinas.

Benchmark	Processador		
	R	M	Z
E	417	244	134
F	83	70	70
H	66	153	135
I	39 449	35 527	66 000
K	772	368	369

- Calcule a métrica de velocidade para cada processador para cada benchmark, normalizada para a máquina R. Ou seja, os valores de razão para R são todos iguais a 1,0. Outras razões são calculadas por meio da Equação 2.5, com R tratado como o sistema de referência. Depois, calcule o valor da média aritmética para cada sistema usando a Equação 2.3. Essa é a técnica utilizada em Heath (1984^{pp}).
- Repita a parte (a) usando M como máquina de referência. Esse cálculo não foi tentado em Heath (1984^{pp}).
- Qual máquina é a mais lenta, com base em cada um dos dois cálculos anteriores?
- Repita os cálculos das partes (a) e (b) usando a média geométrica, definida na Equação 2.6. Qual máquina é a mais lenta, com base nos dois cálculos?

- 2.15** Para esclarecer os resultados do problema anterior, examinamos um exemplo mais simples.

Benchmark	Processador		
	X	Y	Z
1	20	10	40
2	40	80	20

- Calcule o valor da média aritmética para cada sistema usando X como a máquina de referência e depois usando Y como a máquina de referência. Demonstre que, intuitivamente, as três máquinas têm um desempenho relativamente equivalente e que a média aritmética gera resultados enganosos.
- Calcule o valor da média geométrica para cada sistema usando X como a máquina de referência e depois usando Y como a máquina de referência. Demonstre que os resultados são mais realistas do que com a média aritmética.

- 2.16** Considere o exemplo na Seção 2.5 para o cálculo da taxa média de CPI e MIPS, que produziram o resultado de CPI = 2,24 e taxa MIPS = 178. Agora, suponha que o programa possa ser executado em oito tarefas paralelas ou *threads* com aproximadamente o mesmo número de instruções executadas em cada tarefa. A execução é em um sistema de 8 processadores, com cada processador (core) tendo o mesmo desempenho do único processador usado originalmente. A coordenação e a sincronização entre as partes acrescentam mais 25 000 execuções de instrução a cada tarefa. Considere a mesma mistura de instruções do exemplo para cada tarefa, mas aumente o CPI para referência à memória com cada perda de cache para 12 ciclos, devido à disputa pela memória.

- Determine o CPI médio.
- Determine a taxa MIPS correspondente.

- c. Calcule o fator de *speedup*.
 - d. Compare o fator de *speedup* real com o fator de *speedup* teórico determinado pela lei de Amdahl.
- 2.17** Um processador acessa a memória principal com um tempo de acesso médio de T_2 . Uma memória cache menor é interposta entre o processador e a memória principal. A cache tem um tempo de acesso significativamente mais rápido de $T_1 < T_2$. A cache mantém, a qualquer momento, cópias de algumas palavras da memória principal e é projetada de modo que as palavras mais prováveis de serem acessadas no futuro próximo estejam na cache. Suponha que a probabilidade de que a próxima palavra acessada pelo processador esteja na cache seja H , conhecido como razão de acerto.
- a. Para qualquer acesso à memória isolado, qual é o *speedup* teórico de acessar uma palavra na cache ao invés da memória principal?
 - b. Considere que T seja o tempo de acesso médio. Expresse T como uma função de T_1 , T_2 e H . Qual é o *speedup* geral como uma função de H ?
 - c. Na prática, um sistema pode ser projetado de modo que o processador deva primeiro acessar a cache para determinar se a palavra está na cache e, se não estiver, então acessar a memória principal, de modo que, em uma perda (ao contrário de um acerto), o tempo de acesso à memória é $T_1 + T_2$. Expresse T como uma função de T_1 , T_2 e H . Agora, calcule o *speedup* e compare com o resultado produzido na parte (b).

Referências

- a VON NEUMANN, J. *First draft of a report on the EDVAC*. Moore School, University of Pennsylvania, 1945. Reimpresso em *IEEE Annals on the History of Computing*, Nº 4, 1993.
- b BURKS, A.; GOLDSTINE, H. e VON NEUMANN, J. *Preliminary discussion of the logical design of an electronic computer instrument*. Relatório preparado pelo U.S. Army Ordnance Dept., 1946. Reimpresso em Bell, 1971.
- c HAYES, J. *Computer architecture and organization*. Nova York: McGraw-Hill, 1998.
- d BASHE, C.; BUCHOLTZ, W.; HAWKINS, G.; INGRAM, J. e ROCHESTER, N. "The architecture of IBM's early computers". *IBM Journal of Research and Development*, set. 1981.
- e BELL, e NEWELL, A. *Computers structures: readings and examples*. Nova York: McGraw-Hill, 1971.
- f MOORE, G. "Cramming more components onto integrated circuits". *Electronics Magazine*, 19 de abr. 1965.
- g BOHR, M. "High performance logic technology and reliability challenges". *International Reability Physics Symposium*, mar. 2003. Disponível em <www.irps.org/03-41st>.
- h PADEGS, A. "System/360 and beyond". *IBM Journal of Research and Development*, set. 1981.
- i GIFFORD, D. e SPECTOR, A. "Case study: IBM's System/360-370 architecture". *Communications of the ACM*, abr. 1987.
- j STEVENS, W. "The structure of System/360, Part II: system implementation". *IBM Systems Journal*, Vol. 3, Nº 2, 1964. Reimpresso em SIEWIOREK, BELL e NEWELL, 1982.
- k VOELKER, J. "The PDP-8". *IEEE Spectrum*, nov. 1988.
- l BORKAR, S. "Getting gigascale chips: challenges and opportunities in continuing Moore's law". *ACM Queue*, out. 2003.
- m Intel Research and Development. *Architecting the era of tera*. Intel White Paper, fev. 2004. Disponível em <www.intel.com/labs/teraera/index.htm>.
- n GIBBS, W. "A split at the core". *Scientific American*, nov. 2004.
- o ANTHES, G. "What's next for the x86?". *ComputerWorld*, 16 de jun. 2008.
- p NOERGARRD, T. *Embedded systems architecture: a comprehensive guide for engineers and programmers*. Nova York: Elsevier, 2005.
- q GRIMHEDEN, M. e TORNGREN, M. "What is embedded systems and how should it be taught? — Results from a didactic analysis". *ACM Transactions on Embedded Computing Systems*, ago. 2005.
- r KOOPMAN, P. "Embedded system design issues (the rest of the story)". *Proceedings, 1996 International Conference on Computer Design*, 1996.
- s HWANG, K. *Advanced computer architecture*. Nova York: McGraw-Hill, 1993.
- t WEICKER, R. "An overview of common benchmarks". *Computer*, dez. 1990.

- u HENNING, J. "SPEC CPU suite growth: an historical perspective". *Computer Architecture News*, mar. 2007.
- v FLEMING, P. e WALLACE, J. "How not to lie with statistics: the correct way to summarize benchmark results". *Communications of the ACM*, mar. 1986.
- w AMDAHL, G. "Validity of the single-processor approach to achieving large-scale computing capability". *Proceedings, of the AFIPS Conference*, 1967.
- x GUSTAFSON, J. "Reevaluating Amdahl's law". *Communications of the ACM*, mai. 1988.
- y SIEWIOREK, D.; Bell, C. e NEWELL, A. *Computer structures: principles and examples*. Nova York: McGraw-Hill, 1982.
- z BELL, C. MUDGE, J. e MCNAMARA, J. *Computer engineering: a dec view of hardware systems design*. Bedford, MA: Digital Press, 1978.
- aa BLAAUW, G. e BROOKS, F. *Computer architecture: concepts and evolution*. Reading, MA: Addison-Wesley, 1997.
- bb BETKER, M.; FERNANDO, J. e WHALEN, S. "The history of the microprocessor". *Bell Labs Technical Journal*, out. 1997.
- cc OLUKOTUN, K., et al. "The case for a single-chip multiprocessor." *Proceedings, Seventh International Conference on Architectural Support for Programming Languages and Operating Systems*, 1996
- dd HAMMOND, L.; NAYFAY, B. e OLUKOTUN, K. "A single-chip multiprocessor". *Computer*, set. 1997.
- ee SAKAI, S. "CMP on SoC: architect's view". *Proceedings. 15th International Symposium on System Synthesis*, 2002.
- ff BREY, B. *The Intel microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and core 2 with 64-bit extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.
- gg Intel Corp. Intel® 64 and IA-32 Intel Architectures Software Developer's Manual (3 volumes). Denver, CO, 2008. www.intel.com/products/processor/manuals.
- hh SEAL, D., ed. *ARM architecture reference manual*. Reading, MA: Addison-Wesley, 2000.
- ii FURBER, S. *ARM system-on-chip architecture*. Reading, MA: Addison-Wesley, 2000.
- jj Smith, B. "ARM and Intel battle over the mobile chip's future". *Computer*, mai. 2008.
- kk HUTCHESON, G. e HUTCHESON, J. "Technology and economics in the semiconductor industry". *Scientific American*, jan. 1996.
- ll SCHALLER, R. "Moore's law: past, present, and Future". *IEEE Spectrum*, jun. 1997.
- mm BOHR, M. "Silicon trends and limits for advanced microprocessors". *Communications of the ACM*, mar. 1998.
- nn HENNING, J. "SPEC CPU2006 benchmark descriptions". *Computer Architecture News*, set. 2006.
- oo SMITH, J. "Characterizing computer performance with a single number". *Communications of the ACM*, out. 1988.
- pp HEATH, J. "Re-evaluation of RISC 1". *Computer Architecture News*, mar. 1984.